Hochschule RheinMain
Fachbereich Design Informatik Medien
Studiengang Medieninformatik

Abschlussarbeit

zur Erlangung des akademischen Grades

Master of Science

Calibrating LLM based Code Generation

Vorgelegt von	Robin Kuschnereit
am	21. Juli 2025
Referent	Prof. Dr. Adrian Ulges
Korreferentin	Viola Campos

Eigenständigkeitserklärung (gem. ABPO, Ziff. 4.1.5.4 bzw. RPO, §19.6)

Ich bin verantwortlich für die Qualität des Inhalts dieser Arbeit, den ich mit geeigneten wissenschaftlichen Quellen belegt bzw. gestützt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Texte, Gedankengänge, Konzepte, Grafiken, technischen Inhalte und ähnliches in meinen Ausführungen habe ich eindeutig gekennzeichnet und mit vollständigen Verweisen auf die jeweilige Quelle versehen. Alle weiteren Inhalte der Arbeit (Textteile, Abbildungen, Tabellen etc.) ohne entsprechende Verweise stammen im urheberrechtlichen Sinn von mir. Ich versichere außerdem, dass ich die Bachelor-Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich versichere, dass ich KI-Tools lediglich als Hilfsmittel verwendet habe und in der vorliegenden Arbeit mein gestalterischer Einfluss überwiegt. Ich verantworte die Übernahme jeglicher von mir verwendeter KI-generierter Inhalte vollumfänglich selbst. Die jeweiligen Benutzungseingaben (z.B. Prompts) sind der Arbeit beigefügt.

Die vorliegende Arbeit wurde bisher weder im In- noch im Ausland in gleicher oder ähnlicher Form einer anderen Prüfungsbehörde vorgelegt. Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte als Täuschungsversuch gelten und prüfungsrechtliche Konsequenzen haben kann. Insbesondere kann es dazu führen, dass die Leistung nicht bestanden ist und dass bei mehrfachem oder schwerwiegendem Täuschungsversuch eine Exmatrikulation droht.

Wiesbaden, 21. Juli 2025

Robin Kuschnereit

Erklärung zur Verwendung der Masterthesis

Hiermit erkläre ich mein Einverständnis mit den im folgenden aufgeführten Verbreitungsformen dieser Abschlussarbeit:

Verbreitungsform	**Ja**	**Nein**
Veröffentlichung des Titels der Arbeit im Internet	×	
Veröffentlichung der Arbeit im Internet		×

Wiesbaden, 21. Juli 2025

Robin Kuschnereit

Abstract

This work addresses the calibration on LLM generated code. A model is well calibrated when the likelihood of correctness relates strongly with the confidence of the model. The work investigates different approaches to estimate the confidence of generated code samples. One category is the usage of the average token probability. The other ones use an evaluator LLM that assess the code correctness. This is done in a quantitative way where the model is prompted for a numerical confidence value and a qualitative way where the model is prompted for a confidence value from a textual scale. Each value in this scale has a predefined confidence value. The humaneval dataset with the MultiPL-E benchmark is used for the experiments. This makes it possible to translate the original python tasks into 22 different programming languages. The results show that the code samples with the average token probability are not well calibrated out of the box. The evaluator LLM achieves better calibration then the average token probabilities. Furthermore multiple approaches are evaluated to achieve a better calibration. Methods that are used are histogram binning, linear regression, iterative group histogram binning and iterative group linear binning. The simpler methods histogram binning and linear regression returned the best results. Histogram binning achieved an ECE of 0.036 and an ASCE of 0.003. Linear regression achieved the best MSE value of 0.207. The more sophisticated methods iterative group histogram binning and iterative group linear binning also achieved better calibration, but they are not effective as the other methods. This could be the case because they are overfitting. The calibration in specific groups is also explored. Here the **simple** and **combined** grouping style delivered the best results. The **simple** grouping style achieved a ASCE of 0.004. The **combined** grouping style achieved a ECE of 0.053, a MSE of 0.205 and therefore a skill score of 0.077. Furthermore histogram binning and linear regression achieved also the best group calibration and with some exceptions iterative group linear binning was on par with the simpler methods.

Contents

Chapter 1

Introduction

In recent years large language models or short LLM have shown notable abilities in a wide range of tasks. One of these tasks include automated code generation. Models like Chat-GPT, Qwen-Coder or Gemini can produce code snippets, finish function and solve complex programming problems. Although the quality of the generated code has steadily improved, calibration is a critical yet often overlooked aspect. Calibration refers to the correlation between the models confidence in its generated output and the actual correctness of the generated code. The focus of this work lies on the subject Code Generation.

Studies show that these model are already used by software developers [18] and another study finds that approximately 31% of postdocs use AI-assistants like Chat-GPT [9]. For example the developer can utilize the model to generate or refine code through chat interactions. Furthermore the developer can also try to fix faulty code by prompting the model. Vaithilingam et al. [18] show that generated code might not be perfect, but it can be used as a good starting point for further development steps. The AI-assistant builds the rough framework and the developer refines the generated code. This shows that there is great interest in using LLMs and particular in using them to generate code.

Besides the generated textual outputs, the model can return a confidence for each generated token. This confidence value displays the models probability that a this specific token appears at this place in the generated token sequence. An average value of this token probabilities will be used in this work as an indicator for the correctness of the generated code. However this confidence might not be a good indicator to find out if the generate code is functioning or if the model might made some mistakes [11]. Such mistakes could be unreachable code, logical errors or security issues. Therefore calibration could be a solution to make the confidence values reliable.

Calibration plays a crucial role in code generation. It helps the developer to better assess the generated code pieces. Faulty code may lead to system failures, security vulnerabilities or maintenance burdens. A well calibrated model does not only provide helpful code generations but also a reliable metric which reliably determines the uncertainty of the model. This metric should represent how confident the model is in the correctness of its generated code. It is assumed that the confidence score or the token likelihoods are not enough, because the models tend to overestimate or underestimate their results. This fact is confirmed in this thesis and should be improved by using calibration. To achieve this we use different calibration methods. This methods were already use by Detommaso et al. [6]. In their case they used these methods for hallucination detection on question-answer datasets.

This thesis aims to explore the calibration properties of LLM on code generation. The goal of this work

is to investigate methods which assess and improve the calibration of confidence values from a LLM. Furthermore we will show different approaches to retrieve confidence values for code generations. This approaches include: the average token probability and an evaluator LLM which asses the correctness of generated code pieces. This evaluator LLM uses two approaches: qualitative and quantitative. This confidence should represent the expected probability of the correctness after calibration approaches were applied. This means that if a sample has a confidence of 50% it has a probability of 50% to be correct. This helps to build more trustworthy and transparent system which allow the user to identify usefulness of generated code. Furthermore through a reliable uncertainty measure allows the developer to take countermeasures and verify the generated code.

This thesis contributions are the realization of the calibration and multicalibration approaches onto the subject area code generation. Furthermore we look into the subject of creating groups. These groups are used in the multicalibration approaches. Here we look into which groups are useful to better the calibration on code generations. To create this groups we use the metadata of the generated code. Another difference is the fact that we look into the topic of correctness prediction instead of hallucination, what Detommaso et al. [6] did in their work.

This work is structured as follows. In the first chapter 2 the foundations for our work are explained. This includes the basics of LLM, calibration, important metrics and the calibration methods we use. Furthermore we display works that are related to our topic. The next chapter 3 covers an overview over our used components. These components include the data generation, evaluation of code with a LLM and the calibration approaches. Then the evaluation chapter 4 displays the results of our work. Therefore we show the used datasets, benchmarks and models. Furthermore we conduct experiments with the different approaches. This experiments will all answer a question. In the discussion chapter 5 we set our results into context and will display potential weaknesses. Besides that we give some options for further research.

Chapter 2

Foundations

In the following chapter we talk about the foundations we need for our work. We start by explaining the basics of large language models. We briefly look into the training and then the generation process. Furthermore we formally explain the metrics we used to evaluate the calibration on our data. We show an example of bad and well calibrated data and display the scores for each example. After this we look into the used methods and programming frameworks. In the end we check into related work of calibration.

2.1 Large Language Models

The concept of a large language model is a main part of this work. Therefore the origin of these models and how their generations process works are presented. To begin with before the creation of the large language models, language models where used [23]. These language models relied heavily on statistics. Their capabilities were enhanced through the research of transformers [19], especially autoregressive transformers. Transformers are based on multi-head attention. Autoregressive transformers use the before generated sequence to predict the next most likely token. Text is convert into numerical tokens and this tokens are then converted into vectors with help of word embedding. This allows the transformer to learn the context of tokens. Todays LLM are pretrained LLM and have hundred of billions parameters that are learned of massive amounts of text data.

Figure 2.1 displays where the LLM, that are used in this work, are located in the dimension of artificial intelligence (AI) [18]. First we got a big sphere of AI. This sphere represents the discipline of imitating human behavior with machines to solve tasks without explicit instructions. The next sphere represents machine learning. It is a subset of AI and is used to create a model for given input data. This model can then be used to make predictions on new data. Typical algorithms for machine learning are linear regression or support vector machines. The next subset is deep learning. The deep learning models are made of simulated neurons which can be structured in multiple layers. Each of this models got an input layer and an output layer. There may be several hidden layers between the input and output layer. The layers transport information form the input to the output layer. For example this can be used for classification tasks. These deep learning models are trained on large datasets. The training works with the backpropagation algorithm. The last subset in the image shows the generative AI. In our case this are the LLM that are used for the code generation. The generative AI takes an input and can creates, with the help of the input, new content.

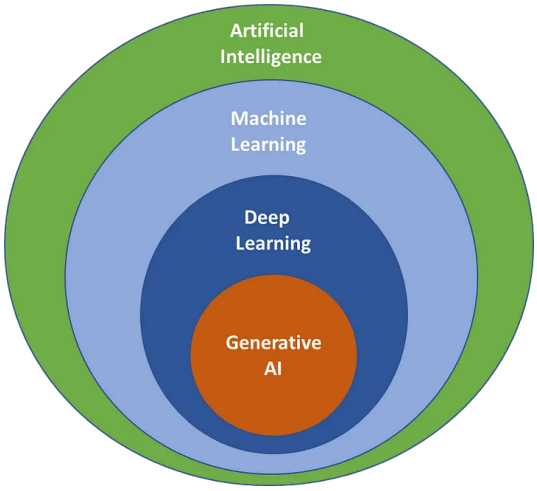


Figure 2.1: The image shows in which subject area generative AI is placed. The image is from Amol Wagh [29]

The goal of an LLM is to predict the probability of a sequence of tokens [22]. This tokens can be words, subwords or characters. After the tokenization each token has an unique numerical value. That number can be used to look up the corresponding context vector. The probability of such a token sequence is calculated with the Equation 2.1. x represents a token and $P(x)$ is the probability that this token is predicted. For a sequence of tokens this results in a chain of conditional probabilities.

$$P(x_0, ..., x_n) = P(x_0) \times P(x_1|x_0) \times P(x_2|x_0, x_1)...P(x_n|x_0, x_1, ..., x_{n-1}) \tag{2.1}$$

To generate new tokens the user inputs context token into the LLM. The generation process is also called decoding, because it uses the decoding part of a transformer. This decoding process can be adjusted with multiple parameters. The context tokens can also be called „prompt“. When the LLM gets the prompt it starts to predict the next token. The model generate new tokens until the maximal token count or the end token is reached. The maximal token count can be defined in the models configuration. For the next token it can be either be the most probable token or the model can consider multiple candidates. Without any randomization the output of such a model would always be the same given the same input [22]. This is also know as greedy search. Therefore it would be deterministic. This is good for task where the outcome is critical, but a disadvantage for tasks where creativity and flexibility are preferred.

Tong Xiao and Jingbo Zhu [22] show two sampling methods in their work. These sampling methods help to get variation into the generated outputs of LLM. The first sampling method is the Top-k sampling. This method chooses the next k most probable tokens during each step of the generation process. After the k tokens are selected the model regenerates the probabilities for these tokens and select the token with the highest probability. The other sampling method is the Top-p sampling. In contrast to the Top-k sampling, Top-p sampling does not choose from a fixed size if tokens. Instead it searches for the smallest subset of tokens that have a cumulative probability greater then the value of p. These methods create a balance between the randomness of the output and the ability to output reasonable sentences.

Another parameter that controls the randomness of the generated output is called temperature [22]. Equation 2.2 shows the softmax function to convert the output logits into probabilities. $P(t_n)$ is the

probability of the n-th token in the vocabulary and l_n is the logit of this token. The sum $\sum_i e^{l_i/t}$ displays all input values. The logit l_n and l_i are divided by the temperature value t. If a higher value is chosen for this parameter then the difference between the output logits will decrease. That has the consequence that the tokens have a more equal opportunity to be selected. Therefore the output has more variation. A lower temperature value has the effect of making the outputs more reliable and even deterministic. This makes it more likely that highly probable tokens will be chosen.

$$P(t_n) = \frac{e^{l_n/t}}{\sum_i e^{l_i/t}} \tag{2.2}$$

as by Matthew Renze and Erhan Guven [14].

This thesis works with the probabilities that come out of this Equation 2.2. They lay the baseline for our experiments.

2.2 Calibration

A model is well calibrated when the accuracy is equal to the confidence of the model [8]. For example calibration plays a role in weather forecast [28]. They calibrate the forecasted weather probabilities with what has been observed in the past. Therefore the can reliably adjust the forecast probabilities that the model makes.

In this work the accuracy is the correctness of the generated code samples. Calibration is important because it helps the end user to rely on the output probabilities. It is not practical to measure calibration over a continuous numerical value. The problem is that probabilities are a continuous numerical value. To counter this problem a uniform grid is used to which the continuous numerical values are assigned.

Figure 2.2 shows the reliability diagram for 4 samples. The model that outputs the confidence for these samples is well calibrated. The x-axis shows the confidence value that was predicted by the model for each bin. In total the diagram shows 11 bins. The y-axis shows the correctness of the samples in each bin. The bin with the confidence 50% has two samples in it. One sample is correct and the other one is incorrect. That is why the bin has a correctness of 50% and is there for perfectly calibrated. The perfect calibration for every bin is symbolized through the blue dotted line. The other two samples are in the bin with 100% and 0%. The one with 100% is correct and the sample in bin 0% is incorrect. This means every bin is perfectly calibrated and therefore the model is well calibrated.

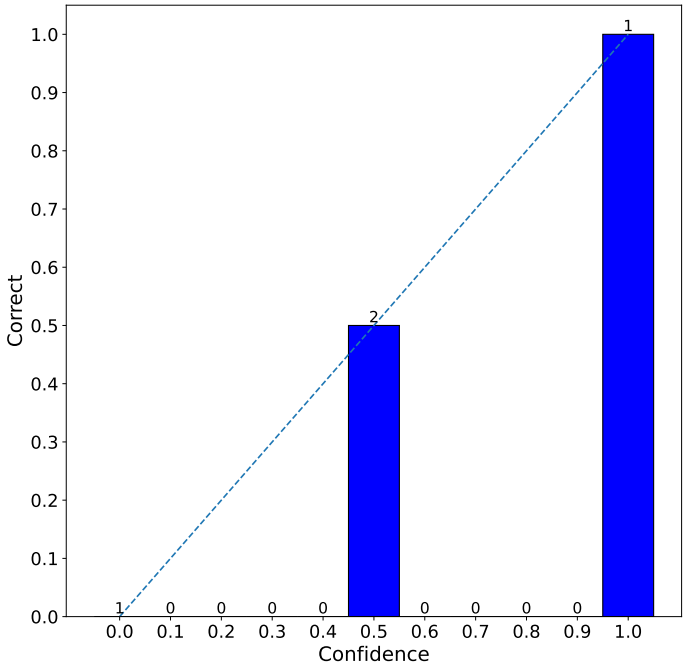


Figure 2.2

On the other side the Figure 2.3 shows the output of a hypothetical model which outputs bad calibrated confidence values. The bin with a 50% confidence is still well calibrated, but the bin 0% and 100% are badly calibrated. That because the sample in bin 0% is correct and the whole bin has now a correctness of 100%. An the opposite is the case for the bin 100%. The sample is incorrect an that is why the correctness in the bin is 0%.

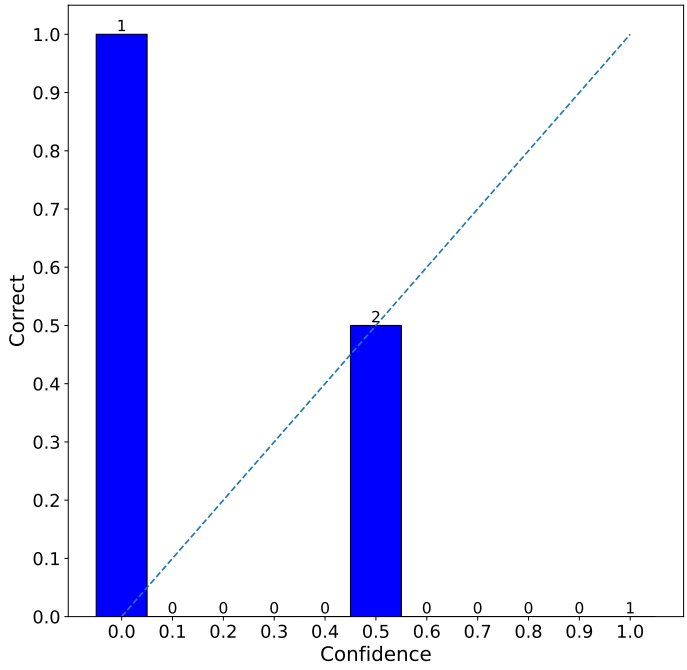


Figure 2.3

This examples will be used in the next Section 2.3 to explain what each score states and how to interpret them.

2.3 Metrics

In the previous section visualizations of calibration where shown. To also quantitatively measure the calibration of a model the following metrics are used.

2.3.1 ECE

The first score is the expected calibration error (ECE) [12]. This score measures the difference between confidence and correctness per bin S_b. S_b contains all samples that are assigned to the bin b. The count of bins is defined by the number m. Each bin is weighted by the share of samples P_b in this bin. The lower the value of the ECE is the better is the calibration of the model. It should be noted that the ECE value is influence by the size of the bins.

$$ECE = \sum_{b=1}^{n} P_b \times |E[y - f(x)|(x,y) \in S_b]|, \text{ where } b \in [\frac{1}{m}] \tag{2.3}$$

For the example of good calibration in the Section 2.2 above the ECE value would be 0, because each

bin has no difference between confidence value and correctness. The bad calibrated example on the other hand has a ECE value of 0.5. It is still no that high because the 50% is still perfectly calibrated.

2.3.2 ASCE

Another metric that is used in this work is the average squared calibration error (ASCE) [6]. It is similar to the previous ECE score. The difference here is that the difference between confidence and correctness per bin S_b is squared and does not use the absolute value. An ASCE of 0 means that the model f is calibrated. The higher this score gets the worse is the calibration of the model.

$$ASCE = \sum_{b=1}^{n} P_b \times (E[y - f(x)|(x,y) \in S_b])^2, \text{where } b \in [\frac{1}{m}] \tag{2.4}$$

The good example calibration in Section 2.2 achieves an ASCE value of zero, which means the model is calibrated. The bad calibrated example on the other hand achieved an ASCE value of 0.5. This is not the worst possible, which is due the fact that bin 50% is calibrated.

2.3.3 GASCE

To measure calibration in specific group of samples it is necessary to have a function $g(x)$ that assigns a given sample to a group g. A group defines a subset of the samples. For example a possible group could be the location for a weather forecast. It could be possible that given the location the predicted probabilities are differently calibrated. The concept of groups will be explained with more detail in the Concept Chapter 3. The group average squared calibration error (GASCE) does the same like the ASCE. The only difference here is that score is calculated for every possible group.

$$GASCE = \sum_{b=1}^{n} P_{b,g} \times (E[y - f(x)|(x,y) \in S_{b,g}])^2, \text{where } b \in [\frac{1}{m}] \text{ and } g \in G \tag{2.5}$$

For this score not exemplary value will be provided because the examples from the previous Section 2.2 had not groups yet. Mainly it works analog to the ASCE. A higher value means worse calibration and a value near zero means that the specific group is better calibrated.

2.3.4 MSE

The mean squared error (MSE) is also known as the brier score [1]. This score is mainly used to measure accuracy. Detommaso et al. [6] state that the MSE is not a direct measure of calibration. They show that the MSE can be broken down into the ASCE and the variance of the model. Through that it is possible that the model with lower MSE can still be not well calibrated.

Equation 2.6 shows how the MSE is calculated. X is the set of samples, Y is the set of labels and n is the count of samples. For every sample the difference between the label and the confidence of the model is calculated. The label can either be 0 or 1, which stands for correct (1) and incorrect (0). The sum is then normalized with the number of samples n.

$$MSE = \frac{1}{n} \sum_{k=1}^{n} (y_n - f(x)_n)^2, \text{where } y \in Y \text{ and } x \in X \tag{2.6}$$

To give some insights into the workings of the MSE here are the values for the good and bad calibration example from Section 2.2. The well calibrated example achieves a MSE of 0.125. The bad calibrated example gives a MSE of 0.625. Therefore lower MSE values can be an indicator for better calibration.

2.3.5 Skill score

The final score that is used to measure the calibration of a given model is the skill score. To specify the skill score it is necessary to introduce the MSE reference score which is defined by Equation 2.7 [16]. To calculate the MSE reference score every sample is put into one big bin. In this bin the probability that a sample is correct $P_{correct}$ is calculated. This is also called a naive estimator that predicts that every sample is correct with the probability $P_{correct}$.

$$MSE_{ref} = P_{correct} * (1 - P_{correct}) \tag{2.7}$$

The skill score equation is displayed in Equation 2.8. Basically the skill score determines the improvement over the naive estimator baseline MSE_{ref} [16]. A negative skill score means that the calibration is worse than the baseline MSE_{ref}. Is the skill score positive it can be a sign of better calibration. A skill score of 0.05 and above is seen as good forecast quality by the Deutsche Wetterdienst [30].

$$\text{skill score} = \frac{MSE_{ref} - MSE}{MSE_{ref}} \tag{2.8}$$

For the good calibration example from Section 2.2 the MSE_{ref} has a value of 0.25 and a skill score of 0.5. This means the calibration is better than the of the naive estimator. The bad example on the other hand has the same MSE_{ref} of 0.25, but a skill score of -1.5 which indicate worse calibration than the naive estimator.

2.4 Methods

2.5 Frameworks

2.6 Related Work

This section present different works that were made in the last years which ave the subject of evaluating LLM generated code.

The work from Spiess et al. [16] deals with the calibration and correctness of code LLM. They argue that these LLM can oft be helpful, but often generate faulty code. They have found out that the from them tested LLM (CodeGen2, Codex, GPT-3.5) are not well calibrated out of the box. This means that the confidence of the LLM does not correlate well with the probability of correctness of the generated results. Furthermore they show how to improve the calibration with platt scaling, which helps to better classify the results.

The work from Ni et al. [11] deals with the ability of LLM to convert language to code. The found that the models which perform best are not the ones that are best calibrated. Better model are according to Ni et al. [11] more reliable, because the calibrated confidence can be used as a good indicator to evaluate the generated code.

Vaithilingam et al. [18] conducted a user study whit software developers. These developers where equipped with the GitHub Copilot assisted to solve programming tasks. The found out that the GitHub Copilot generated correct code in most cases. But the could not discover a significant time saving. They attribute this results to the fact that the developers still need to check the code for correctness and if they find bus they need to first fully understand it to fix the code. Nevertheless the generated code can be helpful to serve as a starting point.

Virk et al. [20] investigate the calibration of confidence score for natural language code summaries. They create a confidence score which mirrors the similarity of the a code summary from a human. To calibrate their results the used platt scaling. They found that there results provide reliable results for the predictions.

The work from Zhong and Wang [24] suggest a new kind of dataset to evaluate LLM generated code. This dataset enables the evaluation of the reliability and robustness of LLM generated code which interacts with APIs. They argument that previous dataset and benchmarks are restricted to small programming tasks, which does not reflect the everyday programming tasks. Even GPT-4 achieves a misuse of 62% of API interactions. This can lead to huge damage in real world applications.

Chapter 3

Concept

Our goal is to create a model f that gets a raw uncalibrated probability and input features for a code generation and outputs a well calibrated probability. That means that we create a model f which estimates a better calibrated probability. This probability is defined by $P(y = 1|x)$, where $x \in X$. X is the set which contains all our code generation samples and Y is the set of labels. If the sample is correct $y = 1$ or incorrect $y = 0$. Such a well calibrated probability helps the software developer to better interpret the code generations and serves as a indicator for possible bugs.

In Diagram 3.1 all main components that are used in our pipeline are shown. The left side shows the code and data generation and the right side displays the calibration part. The process starts with the code generation prompt. This prompt is put into an LLM to generate a program. Besides the generated code we also fetch metadata like the token scores. We process this data to get a raw uncalibrated probability for each generated sample. This serves as a baseline to compare our calibrated probabilities against. After the generation, we evaluate the samples for correctness. This is done by using test cases. After the evaluation, we form groups. This groups are used to summarize samples which have the same properties. For example one group could contain each sample that is written in the programming language python. The concept of groups is used in some of our calibration approaches. Their purpose is to only correct samples that are in a certain group. Therefore we achieve better calibration in this groups.

Furthermore, we will explore an optional process which involves another LLM. This LLM is used to evaluate the generated code. Therefore we select a LLM and input our before generated code with an instruction to evaluate that generated code. This outputs either a percentage of correctness or a qualitative description of the generated code. This LLM evaluation output can either be treated directly as calibrated scores or be used for further calibration. After these step we use the outcoming data for calibration.

After the preparation phase we come to the main part of our work where we can choose between several calibration methods. These methods are histogram binning (HB), linear regression (LR), iterative group histogram binning (IGHB) or iterative group linear binning (IGLB). We measure how well the calibrated scores align with the true probability of a code sample to be correct before each method is started and after the method was used. Besides using every method on their own its also possible to use every method at once. This is used to compare the outputs of each method.

The methods histogram binning, linear regression, iterative group histogram binning and iterative group linear binning are used from the paper multicalibration for confidence scoring in LLMs from

Detommaso et al. [6]

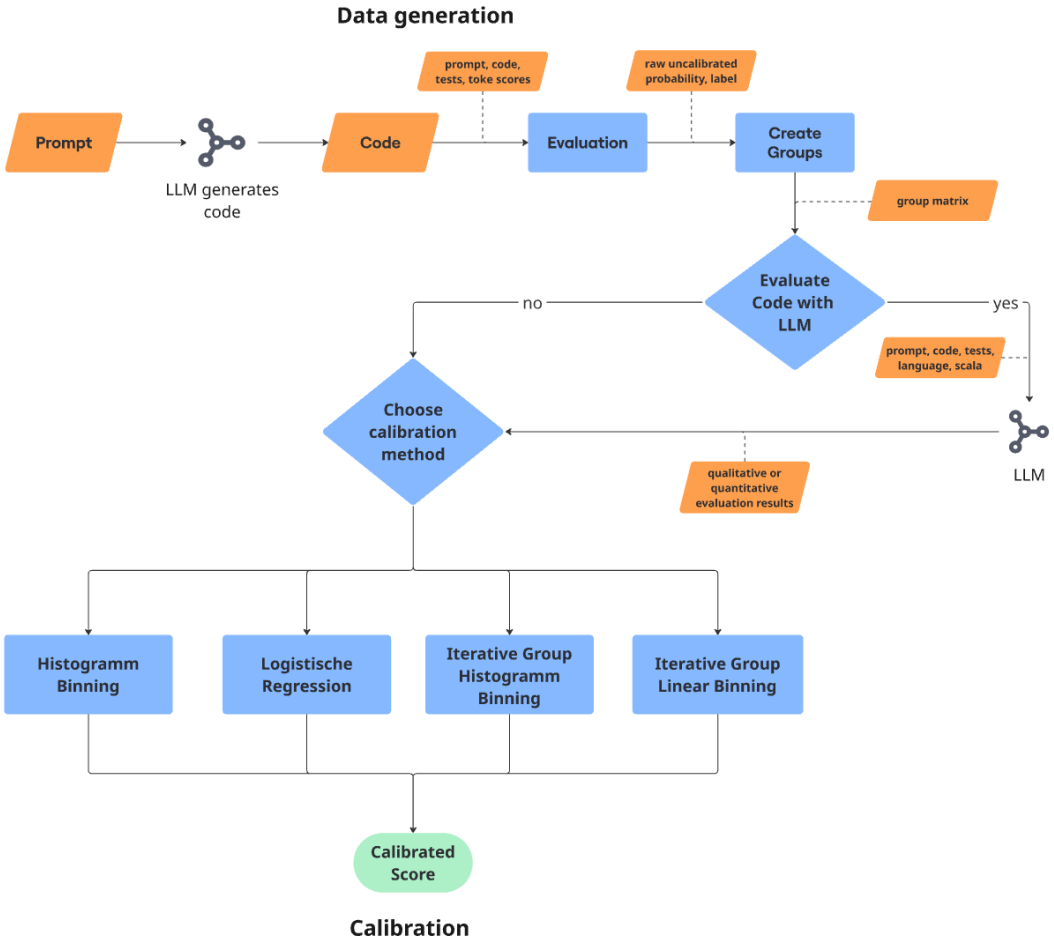


Figure 3.1: Calibration process pipeline. Our process is separated into two main parts. The data generation process and the calibration process. During the data generation we gather all necessary data for our calibration. Which data is gathered can be seen in the orange boxes. For the calibration part we can choose from one of the four approaches or select all to compare them.

3.1 Data generation

At the data generation beginning we got a prompt. This prompt specifies the criteria for the code to be generated. Depending on the dataset, this prompt consists of imports, a method hull and a doc string. In the method hull we find the function name, input parameters with their data type and the expected output data type. The doc string contains a description that specifies the functionality which is expected of the code. In most cases the prompt also contains examples. The examples specify cases fo which inputs which outputs are expected. In Listing 3.1 we can see a example prompt from the humaneval dataset. This prompt is written in python.

```python
from typing import List

def sort_numbers(numbers: str) -> str:
    """ Input is a space-delimited string of numberals from 'zero' to '
        nine'.
    Valid choices are 'zero', 'one', 'two', 'three', 'four', 'five', 'six',
         'seven', 'eight' and 'nine'.
    Return the string with numbers sorted from smallest to largest
    >>> sort_numbers('three one five')
    'one three five'
    """
```

Listing 3.1: Sample prompt from the humaneval python task 19. We can see that the prompt contains an import, the definition of the function with expected output type and a docstring with a description of the task. Furthermore the prompt got an example which shows the model the expected return for a given input. [4]

The prompt is forwarded to a LLM. The LLM then tokenizes, embeds the prompt and predicts the method body. This is done with the selected decoding strategy.

For the generation we use vllm [7]. Vllm is a fast and easy to use framework to conduct inferences with LLMs. Specific LLMs can be selected through vllm's so called model. This can be any model from huggingface hub or a downloaded model. For the generation we pass the prompts into vllm's generate() method. This method also takes sampling parameters. These specify parameters like the temperature, number of generations, stop words, max tokens and logprobs. Which parameters we used during the experiment will be specified in the evaluation Chapter 4.

As response from the LLM, we get the generated token sequences $t_1, ..., t_n$. These sequences contain the completed code pieces. Besides these sequence we additionally fetch details about each token. This data contains an entry for each generated token. This entry contains the token id, the actual log probability and the actual decoded token. The token probability of the i-th token is defined by $P(T_i = t_i | t_1, ..., t_{i-1})$. Furthermore, we fetch the cumulated token probability which sums up all token probabilities. Besides the information about the output tokens we also gather information about the input tokens.

In this work we used open source models. These model allow easy access to token probabilities. These token probabilities indicate how confident the model is in the selection of the token. Note that the returned probabilities are log probabilities and need to be converted into normal probabilities to allow an intuitive assessment. In Figure 3.2 we can see a correct generation for the prompt from Listing 3.1. We can see that the generation was cut of at the first not intended comment, which is located at the last row in Figure 3.2. The colors indicate the confidence of the model in the token: a high confidence is colored dark green and a low confidence is colored in red.

Figure 3.2: The Figure shows the generated code for humaneval task 19 3.1. Each token is highlighted with a color. This color indicates confidence of the model in the specific token. A red color means high uncertainty and a green color means high certainty from the model in the specific token.

The above generation process is applied to every sample in the given dataset, obtaining one generation for each prompt. To summarize, each sample has a prompt, the generated completed code, and details about the token sequence most importantly the individual token scores.

Evaluation

After the generation process we check the correctness of the generated code completions. This is done by applying test cases that are provided with each sample by the respective research dataset. The test cases check if the generated code produces the correct output for a given set of input values. To do so we concatenate the prompt with the generated code completion and append the test cases. Resulting in a program that wen can execute. Depending on the result of the execution it is specified if the sample is correct or incorrect: if it is executable and passes all test cases the sample gets the label correct ($y = 1$). This label is defined as y. Fails the sample test cases or is not even executable it is labeled as incorrect ($y = 0$). We mention at this point that test cases can have blind spots and may not cover every possible input-output combination.

To summarize, we got at this point for every sample the program that consists of the prompt, code completion and the tests, the raw uncalibrated probability and the label which indicates if the sample is correct.

Groups

For some calibration methods we discuss later we need features that describe the sample. We call these features groups. A group is a subset of the dataset. These groups can also be intersecting. For example a piece of generated code is written in the language python and has examples in the initial prompt. We could assume that LLMs have a worse structural correctness and therefore calibration with different kind of programs. Maybe they have better calibration on samples which contain examples and a worse calibration on sample with long input prompts. That would be intuitively the case for humans.

The groups are associated with the samples through a matrix. Let N be the number of samples and M be the number of groups. Then the matrix is N x M. This matrix contains zeros and ones which display the affiliation of the sample and the groups. Each row is a sample and each column represents a group. A zero means that the sample does not belong to this group and a one means it does. This matrix is built during the data loading process and will be used in different calibration approaches in the next sections. A sample matrix is displayed in Figure 3.3. The columns G_m stands for the groups and the row X_n stands for the samples. The set G contains all groups. And the function $g_m(x) = 1$ specifies if the given sample $x \in X$ is in the given group G_m.

$$\begin{array}{c} \\ X_1 \\ X_2 \\ \\ X_n \end{array} \begin{array}{c} \begin{array}{cccc} G_1 & G_2 & \dots & G_m \end{array} \\ \begin{bmatrix} 0 & 1 & \cdots & 1 \\ 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 0 \end{bmatrix} \end{array}$$

Figure 3.3: The group matrix has the size N x M, where N is the number of samples and M is the number of groups. The group function $g_m(x)$ assigns the sample x to a group G_m.

In this work we use three different approaches to create groups:

- The first grouping style we introduce is named „simple“. This grouping style is our the entry point into the group topic. We use simple characteristics to build the groups matrix. The criteria we check for are: the prompt contains the word "example", is the prompt longer than the median prompt, and is the program code longer than the median program code. The idea behind the example group is that it should intuitively be easier to create a program if you have examples that show how the code works. The same goes for the group that checks the prompt length. Here we we assume that the problem becomes more complex the longer the prompt is. For the last group, which checks the length of the generated program, we assume that longer generated code is more likely to be incorrect than shorter code.
- Our second grouping style approach is called „complexity“. For this we analyze the generated programs. This is done with command line tool called „scc“which was created by Ben Boyter [26]. This tool outputs the lines of code, lines of comments and an approximation of cyclomatic complexity per program. The cyclomatic complexity is a metric that measures the independent paths through a program and was introduced by Thomas McCabe [10]. It should be noted that the complexity score we use is only an approximation because its difficult to automate the calculation over multiple different program languages because of their differences in programming paradigms [26]. We choose to create a group for different complexity levels as defined by Thomas McCabe [27]. McCabe [27] introduces a measure of complexity levels which

determine risk. The first group contains all samples that have a cyclomatic complexity of 1-10. These samples are considered to have a small risk. The second group contains all samples that have a complexity of 11-20 and have moderate risk. The third group contains all samples with a complexity of 21-50 and have a high risk. The last group contains all samples that have >50 complexity and have a very high risk. These four groups contain all samples and are therefore self-contained.

- The third grouping style places the samples into groups which represent semantic categories of programming problems. These categories are extracted from the task definition that is contained in the prompt. To achieve this, we check the prompt for certain keywords and add them to the group. The first group is build to represent tasks that use mathematical operations and numbers. This assignment takes place through the check for keywords like 'integer', 'float' and a combination of 'number' and 'calculate'. The second group represents all samples that operate with strings or chars. Therefore we search for keywords like 'string', 'char' or 'word'. The last group contains all samples that use higher level data objects like lists. Here we check for keywords like 'list', 'array' or 'tree'.

3.2 Evaluation through a LLM

One straightforward approach comes to mind to estimate our desired probability $P(y = 1|program)$: We could simply ask another LLM if the generated code is correct. Such auto-evaluation is quite common in question, answering, for example `https://autoevaluator.langchain.com/`.

We also created a method to evaluate the generated code with the help of a LLM. The LLM can either be the same LLM or we can choose another one. This is helpful if want to evaluate the generated code with a more powerful LLM. To get an evaluation of the generated programs, we build a prompt which contains the programming language, the program, which contains prompt, generated code and test cases, and an instruction to evaluate the correctness of the code. The actual prompt templates can be found in the appendix B.1. The prompts and approaches are inspired by the work of Ulmer et al. [17]. Note that the evaluator LLM does not need to be a open source model and it is also not necessary to retrieve the log probabilities. This is possible because we directly use the generated token sequences from the evaluator LLM and do not use the log probabilities.

For the evaluation of the generated code we differentiate between a quantitative and a qualitative approach.

- The quantitative approach tries to directly get a percentage of correctness from the model. Therefore the model is asked to return a percentage of correctness from the range of 0 to 100. This percentage of correctness is then treated as the confidence in the correctness of the sample. We assume that it is hard for a model to rate the program by using a continuous number range.
- The qualitative approach, on the other hand, gives the model the option to choose from a preset of qualitative descriptions. For example „High“or „Very low“. These qualitative scale can be found in Table 3.1. Based on the selection the model makes the sample gets assigned a specific percentage. These values are defined independently and can also be adjusted after the model made its prediction.

For both methods we use template matching to extract the values from the answer token sequences. For the qualitative approach we search for a number with a percentage symbol and for the qualitative approach we search for the predefined qualitative descriptions. Therefore we use the first occurrence.

Description	Probability
Very low	0
Low	0.15
Somewhat low	0.30
Medium	0.45
Somewhat high	0.60
High	0.75
Very high	0.90

Table 3.1: Qualitative scale mapping which is used to rate the correctness of the generation with the evaluator LLM. If the evaluator LLM returns the description low the sample gets a probability of 30%.

The probability outputs of these method can than be used for further calibration methods in the process. It is also possible to not use any calibration method and analyze the pure outputs from the evaluator LLM.

3.3 Calibration foundations

For our calibration approaches we need a probability which reflects the confidence in the generated code. We obtain a first baseline estimate of this confidence from the individual token probabilities. We assume that if the model is „insecure“about its generation these toke score will be low / have a low entropy. With them we calculate the average token log probability per sample. This is done my accumulation of the token log probabilities. This number is then divided by the number of generated tokens. The result is then transformed into actual probabilities by using the exponential function. The formula is displayed in 3.1. We assume that these average probabilities are the confidence of the model in the generated code for further experiments. Lower average token probabilities imply an uncertainty in the model output. Higher confidence scores imply that the model thinks that its output is more reliable. As a results of this process step, we get an probability for each sample that represents the confidence.

$$P^{raw}(x) = exp(\frac{\sum_{k=1}^{n} logprob_k}{n}) \tag{3.1}$$

These uncalibrated probabilities of our code samples are the baseline for our further work. We will use them to verify if we can achieve a better calibration with the calibration methods we display in the following sections.

3.3.1 Discretizing scores

To review the calibration on our generated data, we need to split the probability range into bins. This is done by splitting the range into equally large bins. They are defined by $[\frac{1}{m}] = \{0, \frac{1}{m}, \frac{2}{m}, ..., \frac{m-1}{m}, 1\}$ We control the number of bins with the parameter m. To assign this probabilities to a bin we need to know which probability comes into which bin. To achieve this we calculate the distance to each bin a choose the bin which is closest to the probability. The Equation to calculate said assigned bin can be seen in 3.2. $f(x)$ can be the baseline probabilities of the generated code or the adjusted probabilities

from a calibration approach. For the baseline $f(x)$ would be $P^{raw}(x)$. $f'(x)$ represents the discretized values.

$$f'(x) = argmin(|f(x) - b|), \text{where } b \in [\frac{1}{m}] \text{ and } x \in X \tag{3.2}$$

This mapping is then used to calculate the correctness in each bin b. This is done before and after the calibration method was applied. Through that we can evaluate the calibration on the uncalibrated data, the calibration on the calibrated data and the increase or decrease of metrics that were showed in Chapter 2.

3.3.2 Visualizing calibration result

To compare the approaches we calculate metrics on the uncalibrated data and then after each method was used. Through this we get an idea of how well each method performed under the same circumstances. We can now use we use reliability charts and histograms to plot the changes that each calibration method applies. The reliability charts show each bin and the correctness of the samples in said bin. The histograms show the distribution of the samples in the grid and can therefore track where the distribution mass moved. In Figure 3.4 we can see an example for a reliability chart. On the x-Axis we got the confidence values that we received from the LLM. On the y-Axis we got the probability that a sample is correct for each bin. The blue diagonal line symbolizes a perfect calibration. The bar colors of the reliability diagram display the sample distribution. Darker blue shades mean more samples are in this bin and light blue or white displays less samples. Furthermore the numbers about the bars show the actual count of samples in this bin.

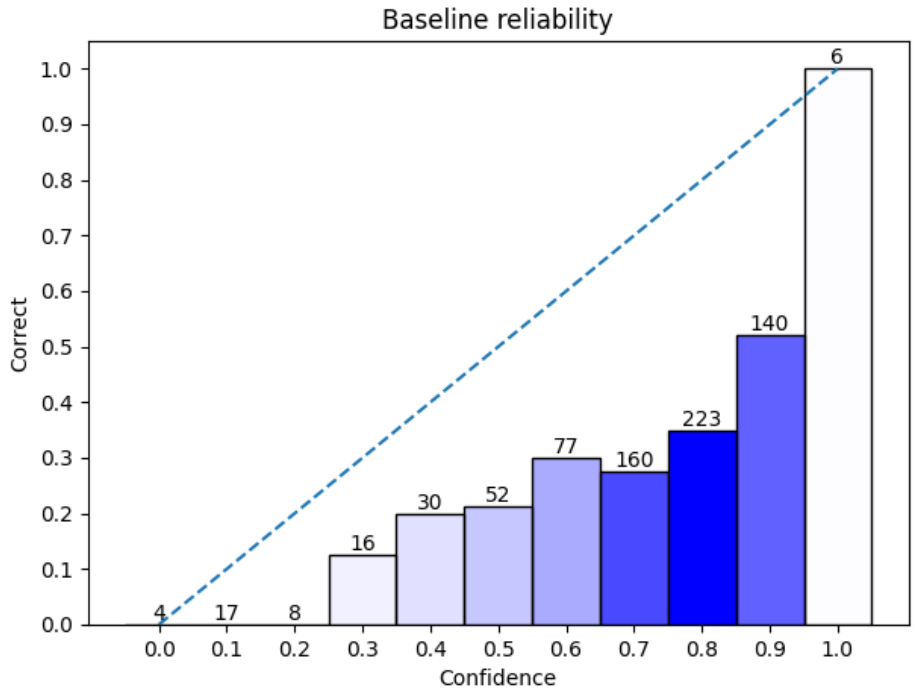


Figure 3.4: Reliability chart to display calibration. The x-axis displays the confidence of the LLM and the y-axis shows the correctness in each bin. Above each bar are the number of samples in the bin. The dotted line indicates perfect calibration. The color gradient shows where the most samples are located. Darker blue means more samples.

In Figure 3.4 we can see the uncalibrated data distribution of a dataset. As we can see the probabilities are unevenly distributed and are not close to the diagonal line which would be the perfect calibration of the underlying data. Perfectly calibrated data would mean that in bin 0.5 are always 50% correct and 50% incorrect samples. The bin with a confidence 0.8 contains 223 samples. It is the bin with the most samples an there for has the darkest shade of blue. The raw uncalibrated probabilities express a high confidence (80%). But we can see that samples in this bin are have a correctness of around 35%. This is an example of poor calibration. For a good calibration the bar should be at 80%.

3.4 Calibration

A calibration method is something that gets the raw uncalibrated probabilities and turns them into better calibrated probabilities. We hope that these better calibrated probabilities provide a better estimation of the correctness of a generated code sample. Some of these methods do not only take the raw uncalibrated probabilities, but also use groups. These groups are used to obtain a better calibration on the subset of samples that are contained in this groups. We hope to achieve better calibration by choosing a good combination of groups.

3.4.1 Histogram binning

The first calibration approach is called histogram binning (HB). This approach is the simplest of those studied in this thesis and tries to adjust all probabilities in a bin at once. To achieve the calibration improvement, we calculate the difference between the correctness and the average confidence in the bin for all defined bins. These so called deltas in formula 3.3 are learned under supervision on a trainings set. Because we discretized the values in each bin the average confidence is always the value of the bin. For example in the bin 0.5 the average confidence is 0.5. The correctness is calculated by using the label of the samples. With the these two values we calculate the difference (delta) between optimal calibration and actual calibration. This is done for every bin.

$$delta_b = E[y - f'(x)|(x, y) \in S_b], \text{ where } b \in [\frac{1}{m}] \tag{3.3}$$

The calculated deltas are then used to update all discretized probabilities in each bin. These updates are made to every defined bin. This is done by adding the corresponding delta with the discretized probabilities in the selected bin. The result of this are new adjusted probabilities for every sample. This process shifts the probabilities to a better calibrated version. But it neglect the fact that different bins and the contained samples are not always disjoint and do maybe depend on each other [6].

3.4.2 Linear regression

The second method we use to calibrate the raw uncalibrated probabilities is linear regression (LR). In this approach the groups appear. The groups are used as input features for the linear regression. In Formula 3.4 these groups are displayed as x_i and can take the value one or zero. As output we want the difference between the label y and the actual probability $P^{raw}(x)$ which is displayed as $delta$. The label y defines if a sample is correct or not and therefore can take the values zero or one. Which means incorrect sample will be shifted into the direction of zero because they become a negative sign and the correct sample will be shifted more to the direction of one because of the positive sign.

$$delta = b + w_1 * x_1 + w_2 * x_2 + ... + w_n * x_n \tag{3.4}$$

To realize this the linear regression learns the weights w_n for each group and the bias b. In this approach we use the actual probabilities and not the discretized values. Further more the bias b of the linear regression is used to adjust samples that have no groups associated with. These weights and the bias are learned on a training set.

With the learned weights $w_1, ..., w_n$ and bias b we predict the adjustment for a given group feature input $x_1, ..., x_n$. The predicted y value is then added to the raw uncalibrated probability of the sample. $f^{LR}(x) = y + f^{raw}(x)$. Through that we get a better calibrated probability for the sample.

3.4.3 Iterative group histogram binning

Iterative group histogram binning (IGHB) is a approach which combines the idea of groups with the concept of bins. Instead of calculating deltas for each bin, like we did it in the histogram binning 3.4.1 approach, we calculate these deltas for every set of bin-group combination. These sets are defined by: $S_{b,g}$, where $b \in [\frac{1}{m}]$, $g \in G$. The deltas are computed by calculating the difference between the correctness Y and the average confidence $f'(x)$ for every set of bin-group combination as we see in Formula 3.5. Through this we create a matrix of delta values. Where M is the number of bins and G is

the number of groups. Then we get matrix with the shape of M x G.

$$delta_{b,g} = E[y - f'(x)|(x,y) \in S_{b,g}], \text{where } b \in [\frac{1}{m}], g \in G \tag{3.5}$$

The problem that arises is that we now have multiple delta values for a sample. That is the case because a sample can be in multiple groups, but only in one bin. So instead of applying the delta to every sample in a bin, like we did with histogram binning, we select one bin-group combination. To calculate which bin-group combination to adjust, we square our deltas from Equation 3.5 and multiply them with the probability $P(S_{b,g})$, where $b \in [\frac{1}{m}]$ and $g \in G$. This probability tells how the samples are distributed over the bin-group combinations. For the formula see 3.6. By doing this we get the bin-group combination which has the highest impact on the calibration of the our data.

$$(b,g) = argmax(delta^2_{b,g} * P(S_{b,g}), \text{where } b \in [\frac{1}{m}] \text{ and } g \in G \tag{3.6}$$

The adjustments of the sample probabilities are made in a loop. This loop stops if the maximal error in the data is smaller then a predefined value α. This α value also defines the grid size in this approach. The α value of 0.1 results in a grid of 10 bins. So the grid is defined by calculating $\frac{1}{\alpha}$ and using this value as the bin width. The max error on the other hand is recalculated on each iteration of the loop. It is defined as the result of multiplying the GASCE with the probability of which a sample is in a given group as we can see in formula 3.7.

$$\text{max error} = argmax(GASCE * P(g)), \text{where } g \in G \tag{3.7}$$

In each iteration we adjust the probabilities on all subsets (train, valid and test set) of our data. For evaluation purposes we track the changes which each iteration made. After each iteration we fit the calibrator with the adjusted probabilities. That means we calculate each deltas again and check if the error is still greater than the α value. Is that not the case we stop the adjustment and the calibration process is finished.

Through the specific adjustment we make for the bin-group combinations there is a high risk of overfitting on the data. That is caused by the small amount of data that is adjusted in each iteration.

To apply the learned changes onto an new sample we need to keep track of every adjustment that is made per iteration. Therefore we need to save the bin-group combination that was adjusted and the delta which was applied. It is important to mention that these changes need to be applied in the exact same order as they where saved. That is because through the changes of time samples are moved into other bins and could be affect by the next change in another way.

3.4.4 Iterative group linear binning

Iterative group linear binning (IGLB) improves the previous IGHB approach. Therefore three adjustments are made. First we not only apply the corrections for exactly one bin-group combination but to cumulative bins instead of individual ones. A bin is now defined like $X <= b2$ not $b1 <= X <= b2$, where b_n is the edge of a bin. This set is defined by the parameter $\otimes$. $\otimes = \leq$ we adjust all values that are smaller than or equal the selected bin. Analog a $\otimes = \geq$ means we adjust all values that are greater than or equal the selected bin. The parameter $\otimes$ specifies this cumulative distribution function. This means our sets $S_{b,g}$ from the iterative group histogram binning are now extended with the parameter $\otimes$. These new sets are defined as follows: $S_{\otimes,b,g}$, where $b \in [\frac{1}{m}]$, $g \in G$, $\otimes \in \{\leq, \geq\}$. The update of a larger number of samples is assumed to help to limit the risk of overfitting.

Secondly we apply so called linear scaling within each bin. This scaling is defined by the formula 3.8:

$$f^{LS}(x) = expit(alpha + beta * logit(f(x)) \tag{3.8}$$

$f(x)$ are the probabilities of the samples and $f^{LS}(x)$ are the new adjusted probabilities. To get an optimal set of alpha and beta values we minimize the MSE 2.6 on Formula 3.8. This is done by using the Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm [21]. As initial start parameter we use 0 for alpha and 1 for beta. By applying the formula we calculate the alpha and beta values for all sets $S_{\otimes,b,g}$. In the prediction phase we select the set which is has to be adjusted. Equation 3.9 shows how the adjustments are only made on a specific subset $S_{\otimes,b,g}$.

$$f^{IGLB}(x) = \begin{cases} f^{LS}(x), & x \in S_{\otimes,b,g} \\ f_t(x) & \text{else} \end{cases} \tag{3.9}$$

We choose the subset, we want to correct, by calculating the probability that a sample is in a given subset with parameter $\otimes$, group and bin ($P(S_{\otimes,b,g})$). We also calculate the delta values like we did in the iterative histogram binning approach. The only difference is that we don not calculate the delta values on exact bins, but on the cumulative ones. The probability $P(S_{\otimes,b,g})$ is then multiplied with the deltas squared. We get the set $S_{\otimes,b,g}$ with the largest potential to improve calibration. This calculation is formalized in the formulas 3.10 and 3.11. Algorithm 1 shows the process of the iterative group linear binning for better understanding.

$$delta_{\otimes,b,g} = E[y - f'(x) | (x,y) \in S_{\otimes,b,g}], \text{where } b \in [\frac{1}{m}], g \in G\ , \otimes \in \{\leq, \geq\} \tag{3.10}$$

$$(\otimes, b, g) = argmax(delta^2_{\otimes,b,g} * P(S_{\otimes,b,g})), \text{where } b \in [\frac{1}{m}], g \in G \text{ and } \otimes \in \{\leq, \geq\} \tag{3.11}$$

The third improvement specifies conditions that lead to an early stopping of the algorithm. The early stopping is also to prevent overfitting. After Detommaso et al. [6] adjustments on too small subset could impact the generalization ability of the approach. Therefore the parameter ϵ is used for early stopping. If ϵ is greater then the probability $P(S_{\otimes,b,g})$ of the chosen subset the algorithm will terminate. The second criterion checks if the MSE is still decreasing on the validation data set. If it stops decreasing or stays the same the algorithm will come to an halt.

To apply the changes onto a new sample, that has not been learned, we need to apply all changes that where made during the iteration onto the sample. The changes need to be in order the where done while training.

Algorithm 1: Iterative group linear binning

Data: $D_{train}, D_{test}, D_{val}$
ϵ
$t \leftarrow 0$
$f_t \leftarrow f'$
Result: f_t
while True **do**
  $mse_t \leftarrow MSE_{D_{val}}(f_t)$;
  $(\otimes, b, g) = argmax_{\otimes,b,g}(delta^2(f_t) \times P(S(f_t)))$ with $b \in [\frac{1}{m}], g \in G$ and $\otimes \in \{\leq, \geq\}$;
  if $P(S_{\otimes,b,g}) < \epsilon$ **then**
    break
  end
  $f_{t+1} \leftarrow f^{IGLB}_{\otimes,b,g}$;
  $mse_{t+1} \leftarrow MSE_{D_{val}}(f_{t+1})$;
  if $mse_{t+1} \geq mse_t$ **then**
    break
  end
  $f_t \leftarrow f_{t+1}$
end

3.4.5 Regressor

In our last approach we choose compare different regressors. With these regressors we again try to predict the correction terms which are used to update the probabilities like in the approach 3.4.2. However we also choose to modify the input features for the regressors.

The new input features consists of the sample groups, the assigned sample probability and a histogram. The groups did not change and are used like we described them in Section 3.1. For the probability we used the average token probability. The histogram was created by gathering the individual token log probabilities. These were transformed into normal probabilities and summarized using a histogram. The histogram bins were divided into ten bins. A sample histogram can be seen in Figure 3.5. We assume that the distribution of token scores can correlate with the correctness of a sample.

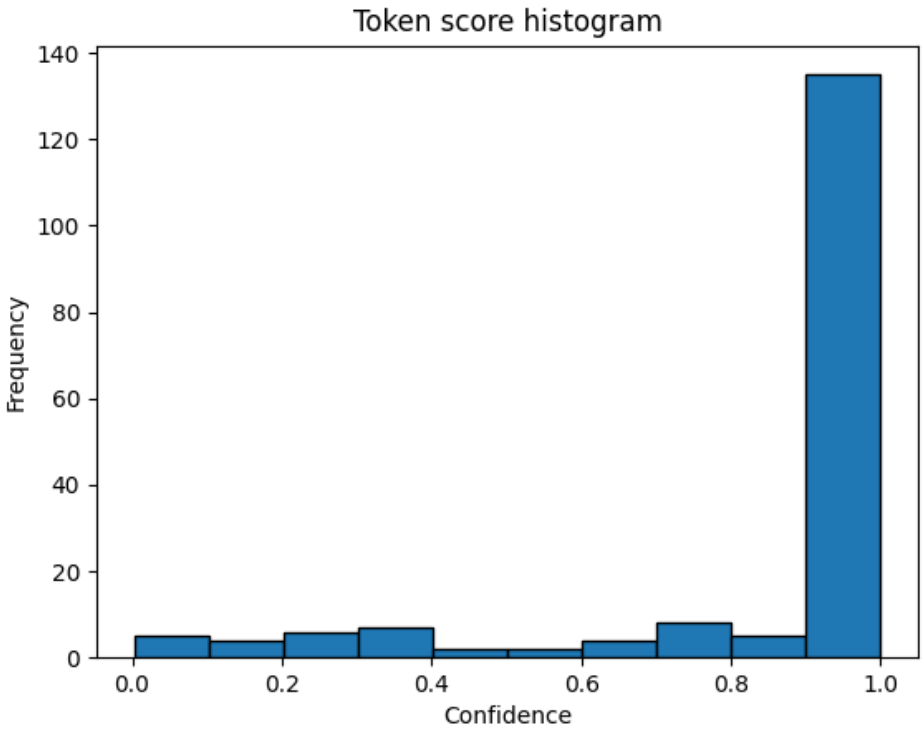


Figure 3.5: Histogram that displays the distribution of the token confidence in the humaneval task 19. The confidence is displayed on the x-axis and the number tokens in each bin are on the y-axis. As we see the model is mostly overconfident in its token selection.

This input features where used to train the regressor. As target we predict the deltas to adjust the probabilities for better calibration. As regressor we use linear regression, support vector regression (SVR) [13, 3] and XGBoost [5].

The linear regression is used like described in Section 3.4.2, but with the modified input features.

For the SVR we use the radial basis function and for XGBoost we used the default settings to fit our data.

All three regressors get the same input features and same target value. We input our features and learn the deltas for calibration. For the prediction part we apply the outputs of the regressor to the sample probability. We assume that the regressors learns deltas that improve the calibration of our data. At this point no bin assignments are made for the calibration process.

Chapter 4

Evaluation

Our goal is to evaluate if the calibration approaches introduced in Section 3 show an improvement in calibration, i.e. yield more reliable estimates of the correctness probability. Furthermore we show which approach provides the best results. We display the datasets and benchmarks used. After this we will display the used models and provide an insight into the parameters used in the generation process and the metrics which are used for the evaluation of the samples. We also conducted experiments which look into some aspects of our calibration approaches. Such as different baselines with the integration of an evaluator LLM. This was mentioned in the Section 3.2. Furthermore we investigate the calibration with different grouping styles. These experiments are shown in Section **??**.

4.1 Data

As data for our experiments we mainly use the humaneval dataset [4]. This data consists of 164 programming tasks in python. Each sample consists of a prompt, which describes the programming task. The prompt contains imports, a method hull and a doc string. An examples prompt is given in Listing 3.1. The tasks are build to assess language comprehension, reasoning, algorithms and mathematic [4].

This 164 samples are too few for our calibration approaches, this will be shown in on of our Experiments 4.5. Therefore we choose to use the MultiPL-E benchmark [2]. The benchmark makes it possible to translate the humaneval tasks into 22 other programming languages. Through this we get a sample count of 3661. The benchmark uses a compiler to translate the test cases, types, doctests and the python terminology. So the evaluation for correctness works like for the python tasks with unit test cases. The samples are evaluated in a docker container. This is because executing generated code can be potentially dangerous. The generations are cut off at a certain stop word. For python this is for example the word „\ndef“which would mark the start of a new high level function.

Generation works over a command line tool where we specify several parameters. These parameters are the model, the dataset (either humaneval or MBPP), the programming language, temperature, batch size, completion limit and our output directory. By default these benchmarks do not return the token log probabilities which we mentioned in the Section 3.1. To return them we made some changes to the existing MultiPL-E scripts. We added the logprob parameter in the automodel_vllm script and adjusted the results files to contain the token ids, log probabilities and the cumulative log probabilities. We generate only one generation for each sample and use batch processing to generate

multiple programs at once.

We train the calibration approaches on 60% of our data. The other 40% are used for validation (20%) and testing (20%). We decided to use cross validation only in certain cases because the data is sufficiently large enough. The validation subset is necessary for the IGLB calibration approach, where it is used for early stopping.

The output of each method is stored in a folder structure. This folder structure is created while loading the data for the calibration approach. It changes dynamically when the approach is called with another parameter set over the command line. The following is a example how the structure looks: `runs/humaneval-all-keep-Qwen2.5\_Coder\_7B-Instruct-1.0-comp-1/lr/linear/avg\_logprob/categories/split` The runs folder contains all results that are generated. The second folder is the run which describes the dataset and for code generation also the model with which the samples where generated. After this the calibration approach is specified in this case its „lr“and under this folder we find the different binning types. For this work we only considered linear binning so its the only possible option. The next option specifies which method is used to generate the probability which is associated with the sample. The options are the average token probability (avg_logprob), the quantitative or the qualitative probabilities which where generated by the evaluation LLM. After this we got folder which specify the grouping style which were explained in the section 3.1. The last folder contains either the results for the partitioned data or for the run with all data and no split into train, test and validation.

In the deepest folder we find charts which display the calibration before and after the method was used and a table which contains the calculated metrics.

4.2 Models and parameters

The model is loaded through vllm and is then used to create the code generations. As model we used Qwen2.5-Coder-7B-Instruct. At the time of this work this is the best performing 7B-Model on the BigCodeBench Leaderboard [25]. BigCodeBench is a benchmark that involves solving tasks with code. It focuses on different functions and complicated instructions.

The model was loaded from the huggingface hub. For the temperature we choose 1 and kept the default value for top-p as 0.95, which means randomized generations. We used one generation per sample.

4.3 Results

In this section we first take a look into the calibration of our baseline. Then we show the results of the different calibration approaches. Therefore we look into the improvement of calibration compared to our baseline. The baseline uses the average token log probabilities per sample. To create our grid we use the parameter $m = 20$. This creates a grid of 101 bins. That means all our probabilities get discretized to values in $0, 0.05, 0.1, ..., 0.95, 1.0$. This grid will be used to calculate the calibration metrics and diagrams for all used approaches. This enables us to compare the approaches later on. We choose the parameter $m = 20$ by doing a grid search.

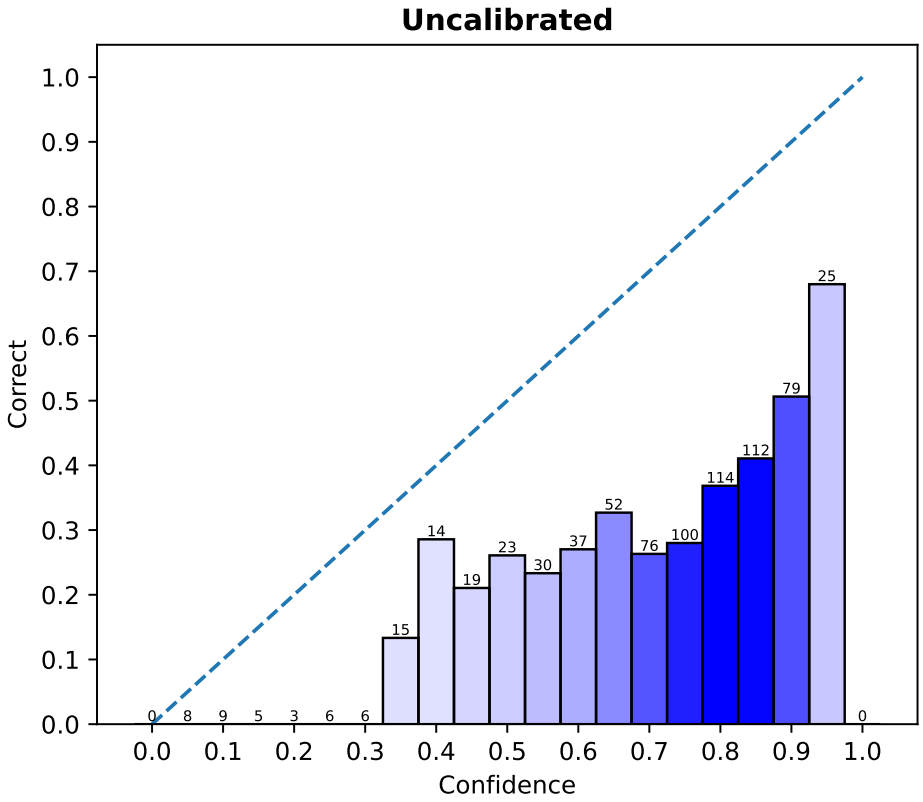


Figure 4.1: Reliability chart that displays the baseline calibration. On the x-axis we got the confidence of the model in the generated code and on the y-axis we got the correctness in each bin. We can see that the majority of the samples (326 samples) are located between the confidence of 75%-85%.

In Figure 4.1 we see the baseline for our following experiments. As we can see the most samples are around the bins with a confidence of 75%-85%. The height of bin 80% indicates that only 40% of the samples are correct. This means that the LLM overestimates the correctness of the code generations. Only the bin with a confidence of 0% has perfect calibration. Every other bin is not well calibrated. For example the 50% bin has only a correctness of around 25%. Which means these samples should be in the bin with a confidence of 25%. In general the bars are below the diagonal line. This means that the correctness of each bin is lower than the actual confidence. Therefore the model systematically overestimates the correctness of the generated code.

ECE ↓	**ASCE** ↓	**MSE** ↓	**MSE$_{ref}$**	**Skill Score** ↑
0.376	0.151	0.355	0.222	-0.599

Table 4.1: Baseline metrics of the uncalibrated average token probabilities. The scores are for the MulitPL-E Benchmark with the humaneval dataset and the Qwen2.5-Coder-7B-Instruct model.

Table 4.1 shows the metrics for our baseline. We can see that the ECE, ASCE and MSE scores are high. The lower these scores are, the better is the calibration. A value of zero for the ECE and ASCE would mean that the model is perfectly calibrated. The MSE_{ref} is a measure of a naive estimator [15]. This estimator places all samples into one bin an measures the correctness in this bin. This value is then used as the confidence for all samples. This estimator is defined by $MSE_{ref} = p_c * (1 - p_c)$. p_c is the probability of correctness over all our samples. The value of 0.222 means that roughly $\frac{1}{3}$ of our samples are correct. The MSE_{ref} will be used to compare our approaches and is part of the skill score. The skill score measures the improvement of the respective calibration via the MSE relative to the MSE_{ref} score. If its positive this indicates an improvement of calibration if it is negative it is a indicator for poor calibration. So we can see that our baseline is not well calibrated an therefore needs adjustments.

Group	**GASCE** $\downarrow$
> Median Prompt Len	0.090
not > Median Prompt Len	0.065
examples	0.038
not examples	0.117
> median code	0.089
not > median code	0.065

Table 4.2: GASCE of the baseline with uncalibrated average token probabilities. The scores are for the MulitPL-E Benchmark with the humaneval dataset and the Qwen2.5-Coder-7B-Instruct model.

Furthermore we got the values for the GASCE in the Table 4.2. The GASCE makes it possible to measure the calibration error for each group independently. For our baseline we only used the „simple“grouping style. In the later experiments we show the effect of other groups. The Table 4.2 shows each group with there name or descriptive feature and the corresponding GASCE value. We can see that the greatest misscalibration is in the group „**not** examples“. And the best calibrated group is the „examples“group. This means that samples with an example are better calibrated out of the box. These leads to the hypothesis that the examples help the LLM to better asses its confidence.

4.3.1 Histogram binning

To recapitulate, the histogram binning method used deltas which correct every sample in a bin. This deltas represent the difference between the correctness and the confidence in the given bin. We used the same parameter $m = 20$ to create the grid like in our baseline. This will be done for all the following approaches. The deltas for each bin where learned on the training subset and applied to the samples in the test subset. We do not mention groups because they do not play a role in this calibration approach.

In Figure 4.2 the results of application of the approach are shown. Left column of the Figure 4.2 represents the test subset before calibration and the right column represents the data after the histogram binning was applied. This kind of diagram will be used for every approach we show in this results section. We see that the distribution of our samples shifted from high confidence values to much lower confidence value. For example before the histogram binning was used the bin 80% contained the most samples. Afterwards the most samples are located in the bin with the confidence value of 45%. Furthermore the data is better calibrated and lines up better with the dotted line, which represents perfect calibration. Nevertheless it did not worked for the test data on all bins as we can see in bin with 95% confidence. This bin has a correctness of 70%. But the samples where shifted into the bin

with 50% confidence. So the approach learned a delta value that does not generalizes well. This could be due the small amount of samples in this bin.

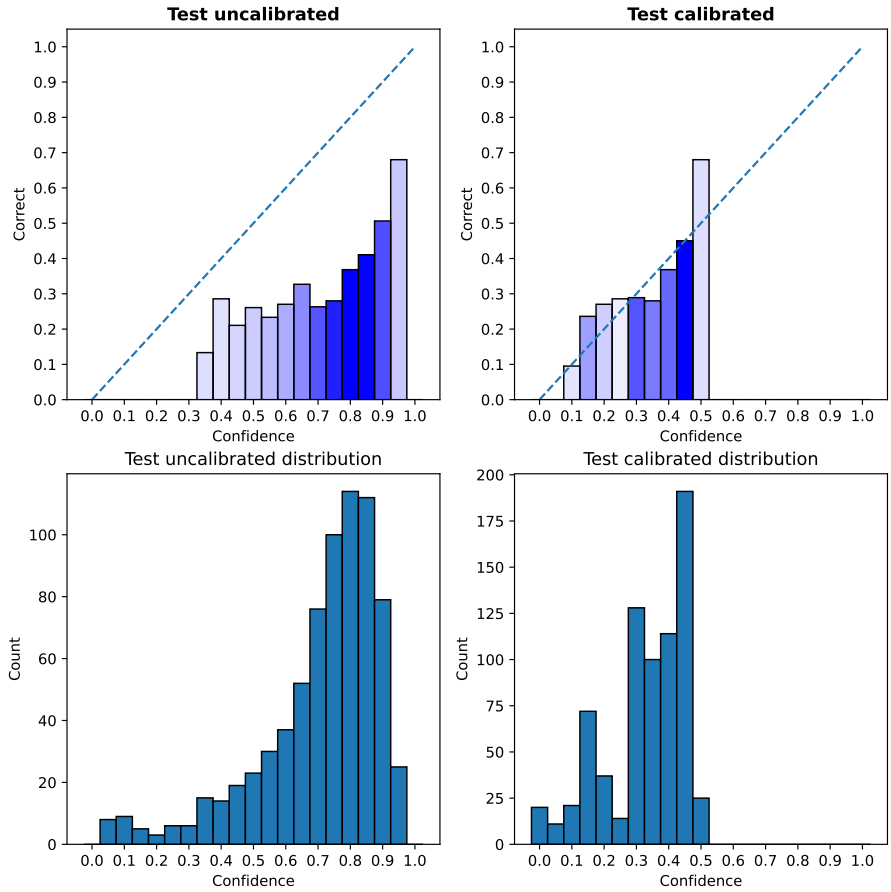


Figure 4.2: Displays the calibration before (left side) and after (right side) the histogram binning approach was used. On the x-axis we got the confidence of the model in the generated code. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution of our samples is displayed.

That the approach improved the calibration can also be seen in the metrics we tracked. They are shown in Table 4.3. The ECE decreased by -0.340 which is a good improvement and ASCE improved to a value close to zero. This assures us that the results are better calibrated then before. Furthermore the MSE decreased by -0.146. Which is not as much as we expected. But Nevertheless the skill score improved by 0.658 and is now positive which indicates a better calibration and an improvement over the naive estimator MSE_{ref}.

	ECE ↓	**ASCE** ↓	**MSE** ↓	$\mathbf{MSE_{ref}}$	**Skill Score** ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
Calibrated	0.036	0.003	0.209	0.222	0.059
Difference	-0.340	-0.148	-0.146	0	0.658

Table 4.3: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

To sum up the learned deltas in the histogram binning approach managed to generalize to the test data. It also achieved a better calibration compared to our baseline with the average token probabilities. Although it does not achieve perfect calibration and still has weakness for bins with low sample count.

4.3.2 Linear regression

Our next method is the linear regression. The goal with this approach is to learn the weights w_n for the defined groups G and a bias b to predict the delta. This delta is then used to correct the raw uncalibrated probability. Note that the corrections are made on the raw uncalibrated probabilities and not the discretized values. This is the first approach which uses the groups as a feature to determine the correction needed. Therefore we take a look into the metrics in Table 4.4 and the GASCE in Table 4.5 to show how the approach improved the calibration in each group.

Figure 4.3 shows the visualized change of calibration. Again the left side shows the uncalibrated test data and the right side shows the data after the linear regression was used. As we see the sample distribution shifted to the left. Generally the calibration of each bin improved but we got some bins where the approach did not generalized well, at least for the view of all samples. In the results for the comparison we also take a look into the calibration for each group individually. Generally we can say that the calibration was improved through this approach but also did not achieve perfect calibration.

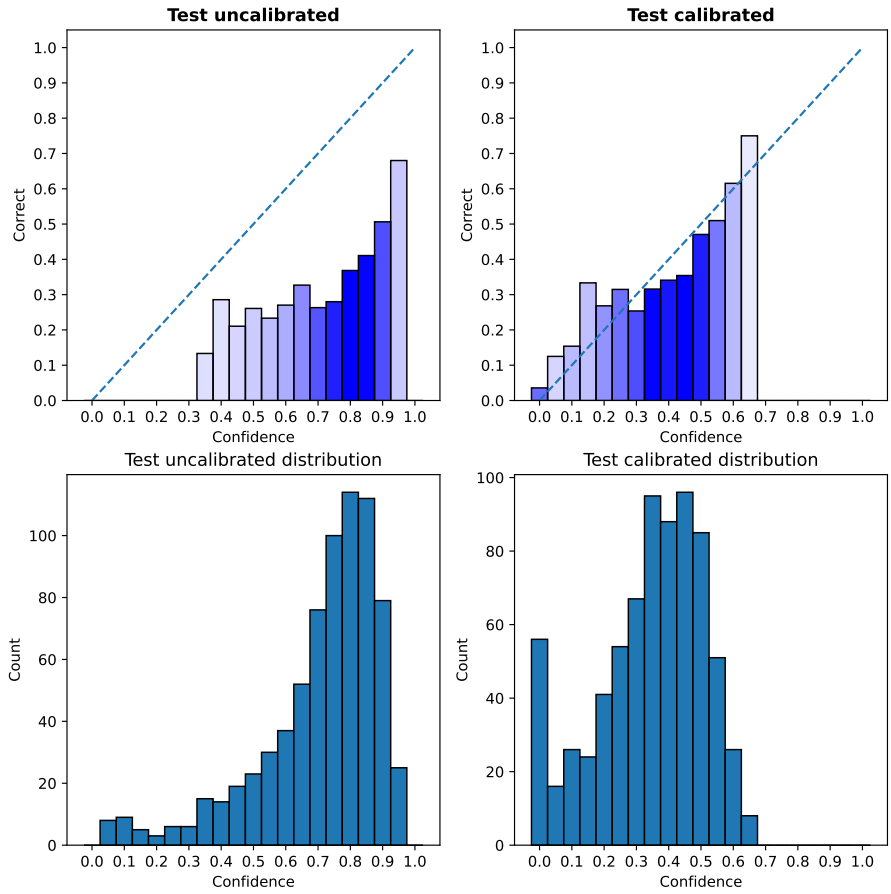


Figure 4.3: Displays the calibration before (left side) and after (right side) the linear regression approach was used. On the x-axis we got the confidence of the model in the generated code. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution our samples are displayed.

If we look into the metrics in Table 4.4 we can see that despite of the visualization the calibration improved quite much on the test dataset. The calibration errors ECE and ASCE improved significantly and also the MSE decreased by -0.148. Also we got a positive skill score that indicates good calibration. Furthermore we can see the GASCE for every group in the Table 4.5. Every group achieved an improvement in calibration and only has an error in the three-digit decimal range close to zero. This means our groups are well calibrated.

	ECE ↓	**ASCE** ↓	**MSE** ↓	$\mathbf{MSE_{ref}}$	**Skill Score** ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
Calibrated	0.057	0.004	0.207	0.222	0.068
Difference	-0.319	-0.147	-0.148	0	0.667
HB	0.036	0.003	0.209	0.222	0.059

Table 4.4: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	**GASCE** ↓	
	Before	After
> Median Prompt Len	0.090	0.001
not > Median Prompt Len	0.065	0.004
examples	0.038	0.001
not examples	0.117	0.004
> median code	0.089	0.002
not > median code	0.065	0.004

Table 4.5: The table shows the GASCE for each group we used. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

To conclude the linear regression approach achieved a good calibration on the overall test dataset. With problems to adjust some samples. Reasons for this could be that the groups are not well chosen. However this opposed by the fact that the GASCE, as we can see in Table 4.5, improved on every group.

4.3.3 Iterative group histogram binning

The iterative group histogram binning used bin-group combinations to make adjustments to the raw uncalibrated probabilities. To use this approach we needed the parameter α which is determined by our parameter m and our stopping condition. For this experiment we chose the value $m = 20$ which means we got an α-value of 0.05. These value was selected after doing a grid search. We got 21 bins and the algorithm comes to a halt when the max error is smaller then 0.05.

We can see in Figure 4.4 that the distribution of our samples shifted again to the left side. This means the actual probabilities where to high and through the calibration process we got lower confidence values in the correctness of our samples. We can see that the calibration improved because the most samples are around the bins 35% and 40% which are reasonably good calibrated. The other bins are not well calibrated. A reason for this could be that the algorithm did not targeted the samples in those bins, because the error is to low through the low sample count in those bins.

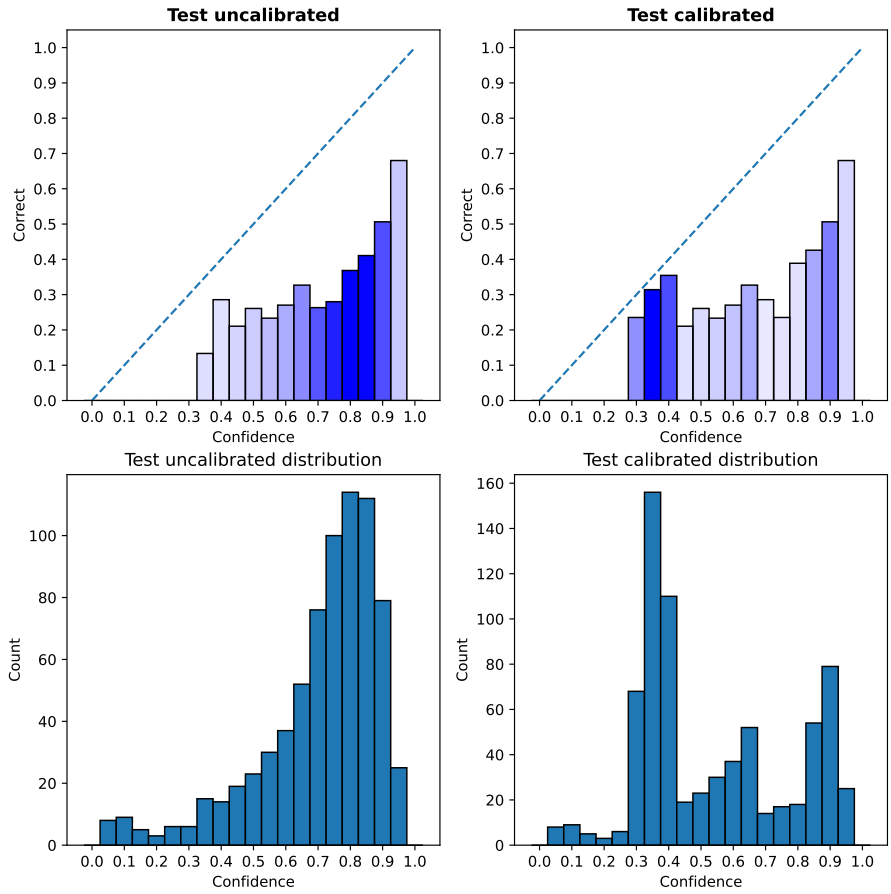


Figure 4.4: Displays the calibration before (left side) and after (right side) the iterative group histogram binning approach was used. On the x-axis we got the confidence of the model in the generated code. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution our samples are displayed.

For further evaluation of the iterative group histogram we take a look into the metrics in the Table 4.6 and Table 4.7. On the ECE we can see an improvement of 0.171. Which means that our bins are have a lower misscalibration then before. But the value is still high. The ASCE and MSE also improved. We can see that the skill score does not reach a positive value. This indicates that the calibration is worse then the naive estimator baseline M_{ref}. Furthermore we see in Table 4.7 that the GASCE decreased on all groups. Especially the „**not** examples“groups achieved a way better calibration.

	ECE ↓	**ASCE** ↓	**MSE** ↓	$\mathbf{MSE_{ref}}$	**Skill Score** ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
Calibrated	0.205	0.068	0.275	0.222	-0.239
Difference	-0.171	-0.083	-0.080	0	0.36
HB	0.036	0.003	0.209	0.222	0.059

Table 4.6: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	**GASCE** ↓	
	Before	After
> Median Prompt Len	0.09	0.042
not > Median Prompt Len	0.065	0.030
examples	0.038	0.033
not examples	0.117	0.039
> median code	0.089	0.037
not > median code	0.065	0.034

Table 4.7: The table shows the GASCE for each group we used. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

We conclude that the approach slightly improves calibration, but has the tendency to overfit on the trainings data. Because of this tendency we decided to test the approach with cross-validation with a 5-Fold. The results showed that the generalization is poor on all folds.

4.3.4 Iterative linear histogram binning

Our last approach is the iterative linear histogram binning. This is a improvement of the previous iterative group histogram binning approach. The main improvements are the adjustment on cumulative bins, the linear scaling in said groups and the early stopping with the MSE on the validation data set and the parameter ϵ. For ϵ we choose the value 0.01. Note that the bin width is again determined by the value m.

The visualized results of the approach can be seen in Figure 4.5. Before the approach was used the most samples have a confidence in the range of 75%-85% and no bin is well calibrated. After the approach was used the most samples have a confidence between 30%-45%. The confidence values were greatly reduced but are now more in line with the correctness value of these bins. The bins with less samples 55%-95% are not well calibrated after the calibration process.

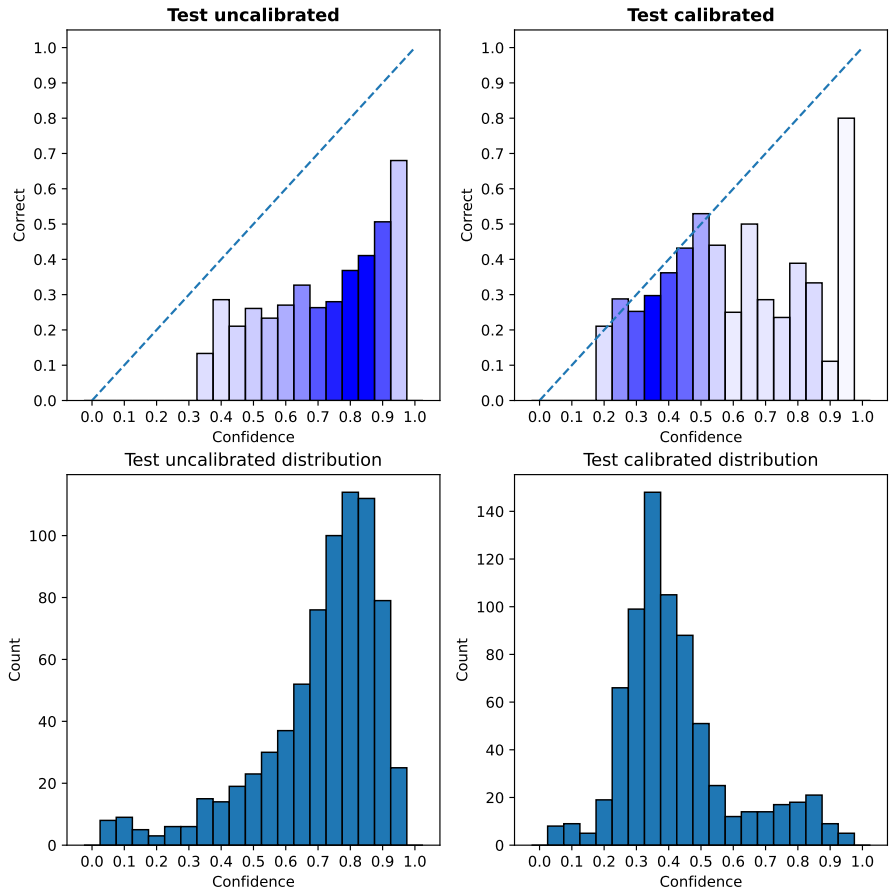


Figure 4.5: Displays the calibration before (left side) and after (right side) the iterative group linear binning approach was used. On the x-axis we got the confidence of the model in the generated code. The y-axis in the upper row displays the correctness in each bin and in the lower row the count of sample in each bin. On the bottom row the distribution our samples are displayed.

The metrics in Table 4.8 show that the calibration has improved. The ECE decreased by 0.275, the ASCE by 0.118 and the MSE by 0.112. The skill score measure shows also an improvement against our raw uncalibrated probability baseline, but it does not achieve a positive value. The GASCE, which can be seen in 4.9 improved on almost all groups. Only the „examples“group has the same value as before. A reason for this could be the small amount of samples (396) in this group. Thereby the max error is to small an the group is not targeted. However the best improvement achieved the „**not** examples“group, which has a GASCE of nearly zero.

	ECE ↓	**ASCE** ↓	**MSE** ↓	$\mathbf{MSE_{ref}}$	**Skill Score** ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
Calibrated	0.101	0.033	0.243	0.222	-0.095
Difference	-0.275	-0.118	-0.112	0	0.504
HB	0.036	0.003	0.209	0.222	0.059

Table 4.8: Shows the metrics that are used to determine the calibration improvement. The metrics for the uncalibrated test data are shown in the first row. The second row shows the metrics after the calibration approach was used and the last row shows the difference between the uncalibrated metrics and the calibrated metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	**GASCE** ↓	
	Before	After
> Median Prompt Len	0.09	0.022
not > Median Prompt Len	0.065	0.017
examples	0.038	0.038
not examples	0.117	0.003
> median code	0.089	0.023
not > median code	0.065	0.016

Table 4.9: The table shows the GASCE for each group we used. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

To summarize the iterative group linear binning approach achieved a better calibration against our baseline. Nevertheless it should be mentioned that the strong improvement of only one group is an indicator that the algorithm came quickly to a halt. Reasons for that could be the stagnating MSE on the validation set or an improper ϵ value.

4.3.5 Comparison

To summarize we now compare the approaches and check which one achieved the best calibration results. Therefore we first compare them visually with each of the reliability diagrams as above. This can be seen in Figure 4.6. Then we look into the calibration which is achieved inside each group and check the metrics for each method.

Figure 4.6 shows the reliability diagrams for each method. Confidence on the x-axis and correctness on the y-axis. The chart in the first row displays the baseline calibration on the test subset. The second row contains the histogram binning and linear regression calibration charts. The last row the iterative group histogram binning and iterative group linear binning results.

We can see that visually the histogram binning and linear regression approaches show the best calibration results. The iterative group histogram binning approach on the other hand looks slightly better calibrated, but worse then the other approaches. The improved version, the iterative group linear binning, shows again better results then the iterative group histogram binning approach but worse then histogram binning and linear regression.

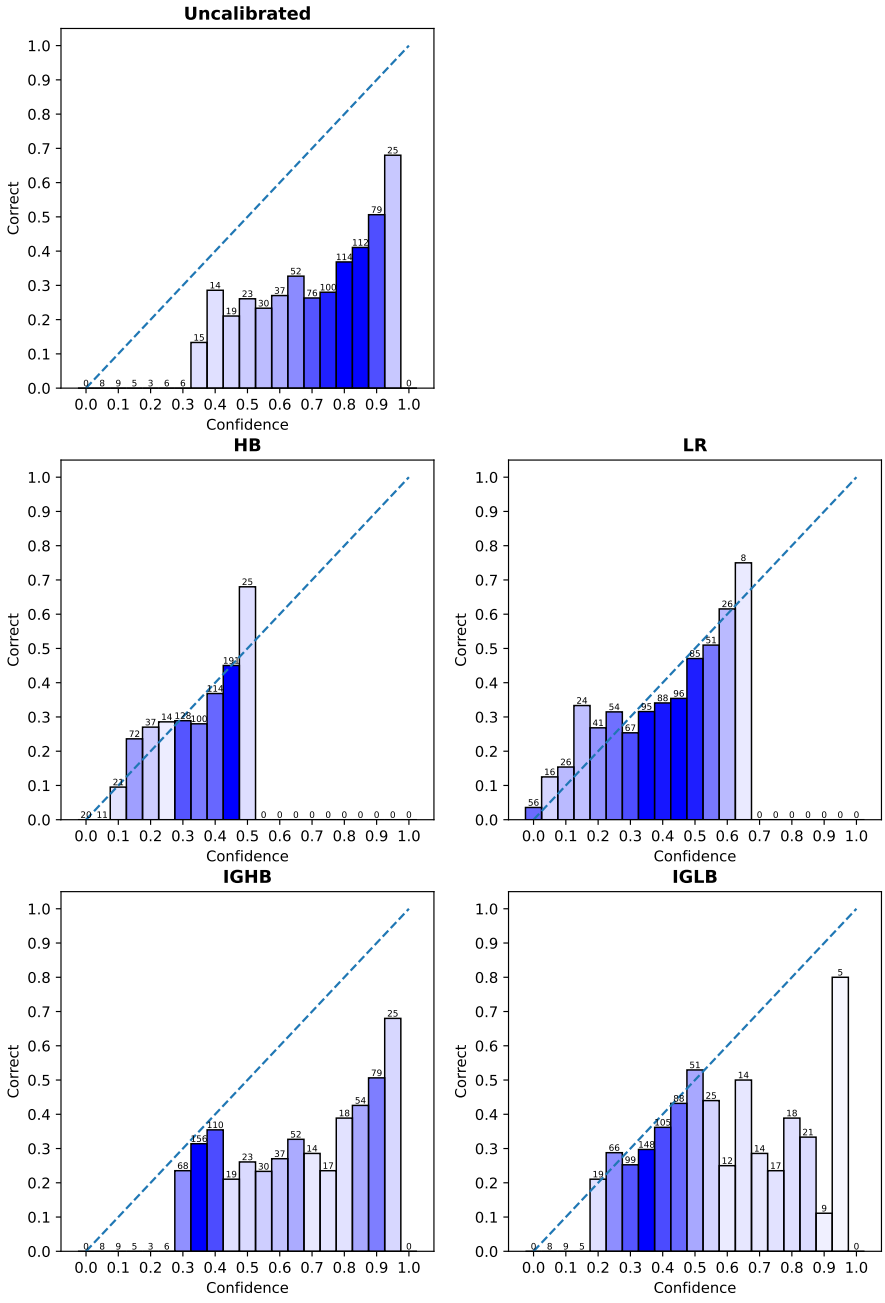


Figure 4.6: Displays the calibration before and after each approach was used. On the x-axis we got the confidence of the model in the generated code. The y-axis displays the correctness in each bin. Over each bar the samples count is displayed. The first reliability chart shows the uncalibrated data. The other charts stand for each approach we used.

What we see visually also reflects in the metrics which are displayed in Table 4.10. The uncalibrated row stands for the baseline with average token probabilities. All other rows are the corresponding approaches we already described in previous sections. We can see that the simple histogram binning approach works the best on the metrics with lowest ECE and ASCE. Not far off comes the linear regression which was the first approach that used groups to achieve a better calibration. Linear regression achieves the lowest MSE and therefore the best skill score. The worst performance has the iterative group histogram binning with highest ECE, ASCE, MSE scores and lowest skill score. That is the case of the poor generalization ability and overfitting on the training dataset. The improvements that where introduced with the iterative group linear binning have enhanced the metrics as we can see in the last row. Nevertheless the improvements were not enough to beat the histogram binning or linear regression approach.

Method	**ECE** ↓	**ASCE** ↓	**MSE** ↓	**MSE_{ref}**	**Skill Score** ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
HB	**0.036**	**0.003**	0.209	0.222	0.059
LR	0.057	0.004	**0.207**	0.222	**0.068**
IGHB	0.205	0.068	0.275	0.222	-0.239
IGLB	0.101	0.033	0.243	0.222	-0.095

Table 4.10: Shows the metrics for each approach we used for comparison. The bold scores indicate the best value for said metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

Group	**GASCE** ↓				
	Uncalibrated	HB	LR	IGHB	IGLB
> Median Prompt Len	0.09	**0.001**	**0.001**	0.042	0.022
not > Median Prompt Len	0.065	0.005	**0.004**	0.030	0.017
examples	0.038	0.003	**0.001**	0.033	0.038
not examples	0.117	**0.003**	0.004	0.039	**0.003**
> median code	0.089	**0.002**	**0.002**	0.037	0.023
not > median code	0.065	**0.003**	0.004	0.034	0.016

Table 4.11: The table shows the GASCE for each group we used. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group.

The next Figure 4.7 is structured the same as Figure 4.6. The charts show the calibration per group. Each circle represents one group. The circle diameter indicates the number of samples in the group. The assignment is as follows:

- > Median Prompt Len ●
- **not** > Median Prompt Len ●
- examples ●
- **not** examples ●
- > median code ●

- **not** > median code ●

On our baseline every group has an approximated confidence of 70%. These confidence values are the average confidences per group. Every groups is far off from being well calibrated. In the diagram for the histogram binning approach we can see that the groups where just shifted to a lower confidence value. This makes sense because we did not use the groups in this approach. The linear regression on the other hand achieves a solid calibration for nearly every group. The yellow and blue group are slightly off the perfect calibration line. Furthermore the confidence shifted from 70% to a range of 25%-40%. Iterative group histogram binning betters the calibration on nearly every group except the green group which has less samples then the other ones. It is expected that the group with low sample count has a worse calibration, because the error in this group will be lower. That is because the probability of each group plays a role in calculating said error. The improved iterative group linear binning approach achieved better calibration than its predecessor, but also does not succeed in correcting the green group. We note that the probability distribution in each group can vary and could contain information of the group calibration.

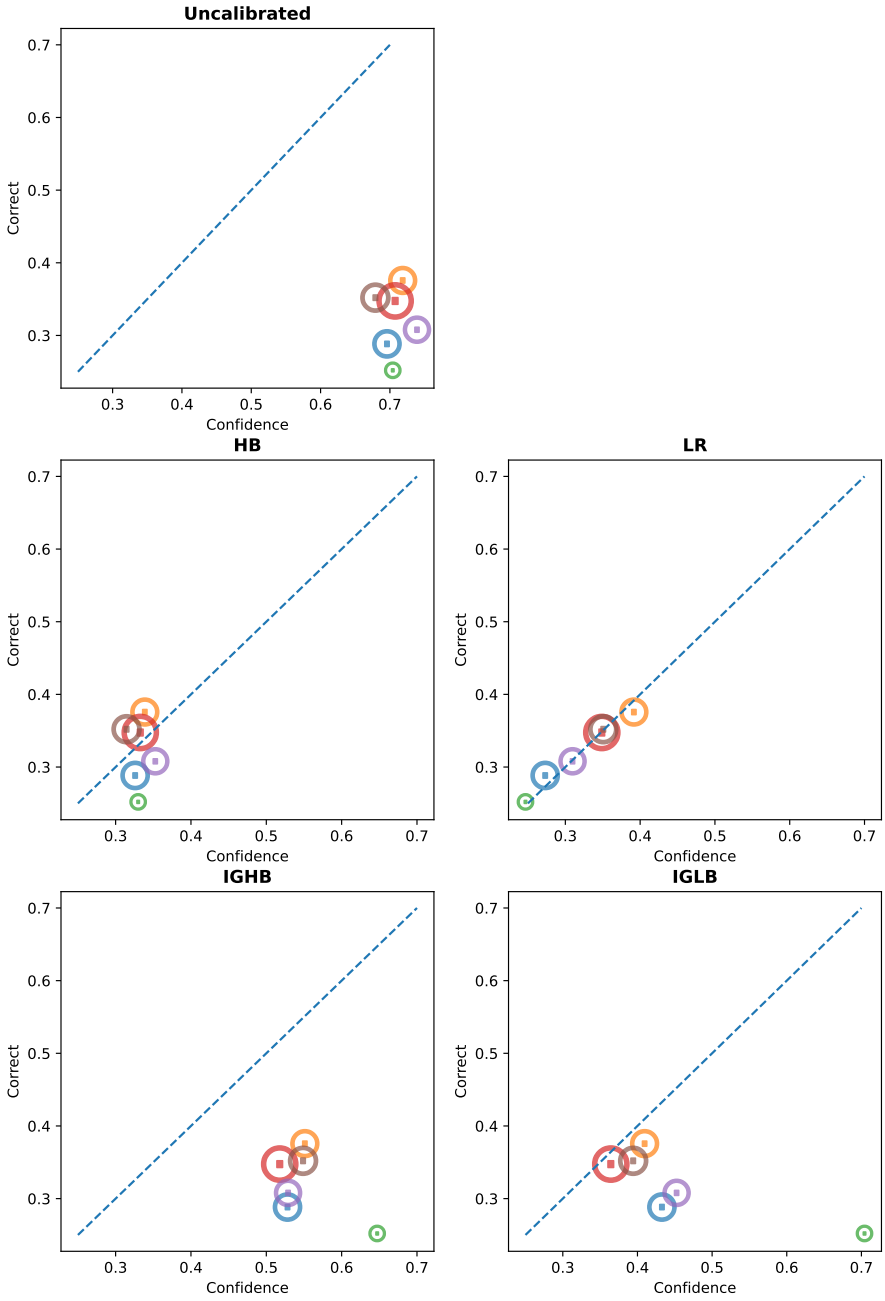


Figure 4.7: Group calibration for the uncalibrated data and then for each approach we used for calibration. Each circle stand for a group. The size of the circles indicate the amount of samples. Confidence is on the x-axis and the correctness of the group is in the y-axis.

To conclude we can say that the simple approaches histogram binning and linear regression delivered the best calibration results. And the iterative group histogram binning and iterative group linear binning approach performed worse but not quite bad. Furthermore the linear regression delivered the best calibration for the groups.

4.4 Evaluator LLM

Now that we have an overview of the results of the average token probability baseline, we want to explore two more ways to get an initial probability of correctness for a sample. Therefore we look into two approaches that utilize an LLM to estimate code correctness: quantitative and qualitative evaluation of the code generation. In the quantitative approach we prompt an LLM with the generated code and ask it to assign a numerical probability of correctness for the prompted code. The qualitative approach gives the model a scale of quality categories (see Table 3.1) and the LLM selects one of the predefined measures.

The model we choose as evaluator LLM is the „gpt_4o_mini“model from OpenAI. The research question for this section is:

> RQ 1: How well does gpt-4o-mini estimates the correctness of the generated code?

Quantitative

We start with the quantitative approach. Figure 4.8 shows the uncalibrated quantitative evaluation results. Uncalibrated because it is possible to calibrated the results with the investigated methods. With the darker blue shades we can see that the most samples (500) are in the range of 85%-95%. The bin with a confidence of 95% has a correctness of only 50% and is therefore not well calibrated. Below the confidence of 55% are only a few samples and none of them are correct.

Generally we can say the the model is overconfident in its evaluation and does not achieve a well calibrated distribution. A reason for this could be that the LLM is not good in generating numerical values to express a probability.

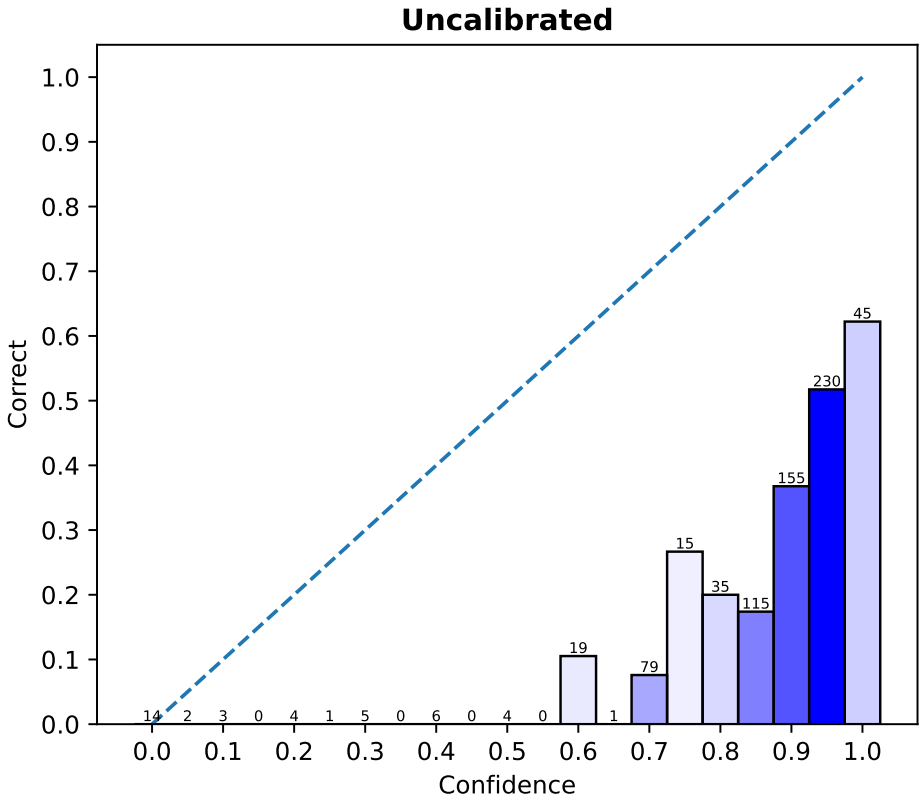


Figure 4.8: Diagram display the confidence on the x-axis and the correctness per bin on the y-axis. The dotted line represents perfect calibration. All correct samples are above the confidence of 55%. The most samples are located around the confidence of 95%. The quantitative results are not well calibrated.

Qualitative

On the other hand we got the qualitative approach. Here use the LLM to select an option from a quality scale. Each option has a specific probability assigned. Figure 4.9 shows the reliability diagram for the qualitative approach. The samples are distributed in bins and are apart by 15%. This is the case because of the assignment with our scale values, which we showed in Table 3.1. Furthermore we see that the higher the confidence is the higher the correctness of the bin gets. An exception is the bin 15% where we got no correct samples. The most samples are located at the confidence values of 30%/75%/90%.

Visually the calibration looks better then the calibration of the quantitative approach and the average token probability baseline. But it is still not well calibrated as the each bar is below the perfect calibration line.

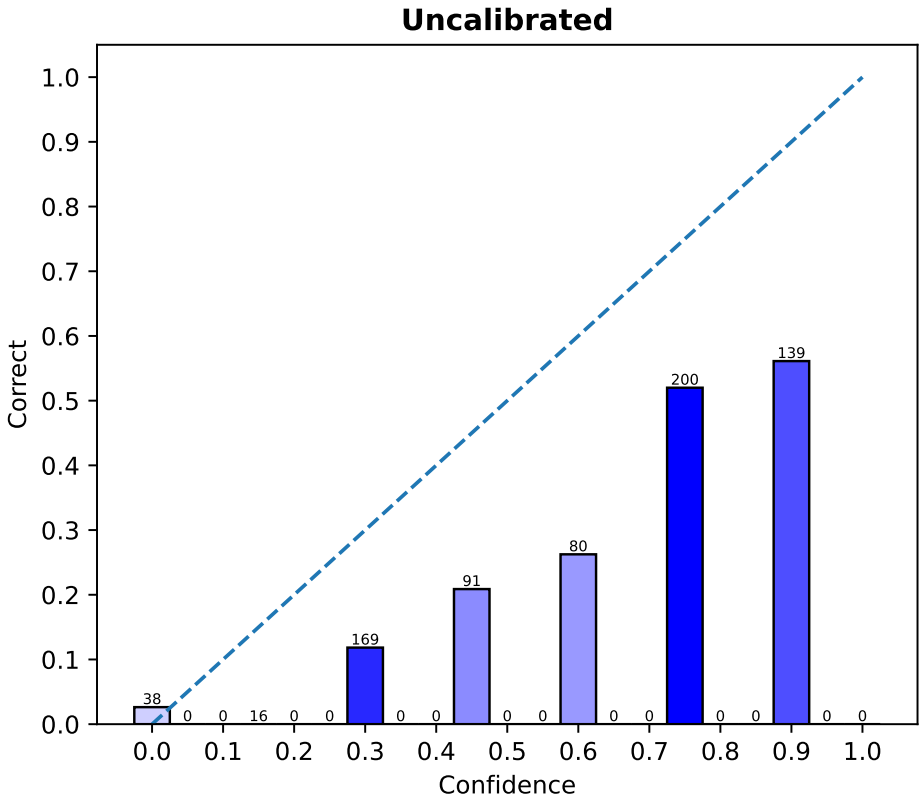


Figure 4.9: Diagram display the confidence on the x-axis and the correctness per bin on the y-axis. The dotted line represents perfect calibration. The distance between the bins is 15%. The reason for this is the qualitative scale 3.1 we used.

Metrics

Visually we showed that the quantitative approach is better calibrated out of the box then the average token probability baseline and quantitative approach. Table 4.12 shows the metrics for the baseline and the two evaluation approaches. The quantitative approach has the highest ECE, ASCE and MSE values. Also the skill score is higher. Therefore we can say that the quantitative approach is the worst calibrated of the three. The average token probability baseline is better calibrated out of the box then the quantitative evaluation. It achieves better scores on all metrics. The best calibrated approach is the qualitative one. We can see that it achieves the best scores on all metrics. Especially the ASCE is surprisingly low.

Baseline	ECE ↓	ASCE ↓	MSE ↓	$\mathbf{MSE_{ref}}$	Skill Score ↑
Avg. Token Prob	0.376	0.151	0.355	0.222	-0.599
Quantitative	0.507	0.274	0.460	0.222	-1.072
Qualitative	**0.240**	**0.0640**	**0.246**	0.222	**-0.108**

Table 4.12: Shows the metrics for each approach and the average token probability baseline. The bold scores indicate the best value for the shown metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

To conclude we looked in the different approaches with visualizations and metrics to answer our research question RQ1. We showed that the qualitative approach is the best calibrated out of the box. But it is still worse then the MSE_{ref}.

4.5 Dataset Considerations

This experiment shows the results of our calibration approaches. All calibration approaches are tested on the original humaneval dataset (only python) and with only one generation per sample. Note that the original dataset has 164 programming tasks. Therefore we will have 164 samples which we split into train, test and validation subset. This splitting results in 33 samples for our test dataset.

> RQ 2: Is the original humaneval dataset (164 samples) enough to achieve a reasonably good calibration?

Figure 4.10 shows the uncalibrated test data in the first row. We see that there is no clear increase in correctness from the left x-axis to the right x-axis. Most of the bins show a correctness of 50% and only have two samples. The most samples (7) are located at the confidence of 80%.

Visually none of the displayed calibration approaches achieve a good calibration. After the calibration approaches were applied the most bins have a correctness above the perfect calibration line. This means that the samples in this bin are now underestimated in their correctness. For example the bin with 30% in the histogram binning approach has a confidence of 30% but an actual correctness of around 65%.

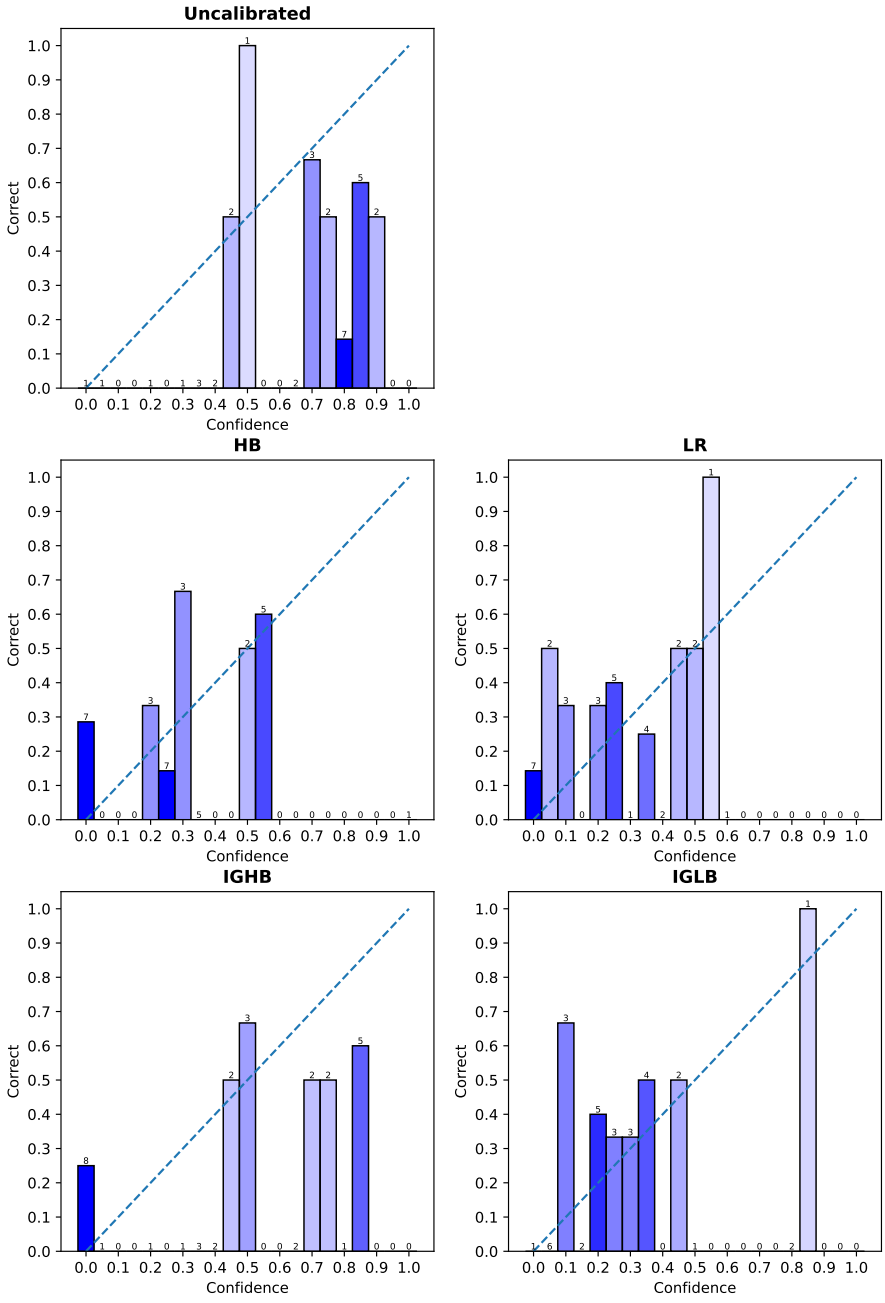


Figure 4.10: Displays the reliability charts for the avg token probability baseline and the calibration approaches. We can see that none of the approaches achieve a visually good calibration.

To be sure we that the calibration did not generalize well, we look into the metrics which are displayed in Table 4.13. The ECE and MSE improved slightly on all approaches. The ASCE also improved but more than the ECE. Generally the linear regression approach provides the best results for the restricted amount of data.

Method	ECE ↓	ASCE ↓	MSE ↓	$\mathbf{MSE_{ref}}$	Skill Score ↑
Uncalibrated	0.350	0.173	0.301	0.211	-0.427
HB	0.220	0.083	0.244	0.211	-0.156
LR	**0.194**	**0.057**	**0.228**	0.211	**-0.081**
IGHB	0.280	0.103	0.251	0.211	-0.190
IGLB	0.200	0.088	0.230	0.211	-0.090

Table 4.13: Shows the metrics for each approach we used for the original humaneval dataset. The bold scores indicate the best value for said metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

To summarize visually we cannot identify a strong calibration improvement. The metrics of the approaches show a gentle decrease against the baseline calibration. We conclude that the amount of samples in the humaneval dataset with one generation per programming task is not enough to create a good calibration model. A reason for this could be the bad generalization on the small dataset, which can be seen in the reliability charts for all used calibration approaches. Only one bin per approach looks like it is well calibrated. This suggests that achieving better calibration results would require more data, even compared to our main dataset.

4.6 Grouping Criteria

In this experiment we want to evaluate the impact of different grouping styles. We defined this styles in Chapter 3. To recapitulate we got the simple grouping style, complexity and categories.

- The **simple** grouping style checks if the prompt is longer than the median prompt length, if the prompt contains the word „example“and if the code is longer than the median code.
- With the **complexity** grouping style we classify each program into categories of how complex the generated program is.
- The **categories** grouping style assigns the code groups by checking for certain keywords. For example checking for the keywords like 'integer', 'float' and a combination of 'number' and 'calculate'.

We want to answer the research question:

RQ 3: Which grouping style provides better calibration?

Therefore we display the results for the complexity and categories grouping style. The results of the simple grouping style where already displayed in the previous results Section 4.3. For the sake of completeness the diagram that displays the calibration per used calibration approach for each grouping style can be found in the Appendix A.1, A.2. In the follow Section we look at the metrics, group calibration and GASCE values for both grouping styles.

Complexity

Figure 4.7 shows the calibration for each complexity group. Each circle represents one group. The circle diameter indicates the number of samples in the group. The color assignment is as follows:

- Cyclomatic complexity: 1-10 ●
- Cyclomatic complexity: 1-20 ●
- Cyclomatic complexity: 21-50 ●
- Cyclomatic complexity: >50 ●

We see that the groups are not well calibrated in our baseline. This is displayed in the diagram in the first row. The blue group contains the most samples, where the red group is almost invisible. All groups are located around the confidence of 70%-80%.

The histogram binning calibrated the blue and yellow group almost perfectly. The groups with a low sample count (green and orange) are not well calibrated. The linear regression calibrated the blue, yellow and green group better but also did not achieve good calibration within the red group. After both approaches the yellow and blue group have a confidence of 30%-40% and the correctness is also in that range.

Iterative group histogram binning only adjusted the values for the blue group. But the adjustment was not enough to reach the perfect calibration line. The other groups were not adjusted. The reason for this can be the low sample count and therefore low max error.

The improved iterative group linear binning did a better job calibrating the blue group. This group is well calibrated. But it also fails to correct the other groups.

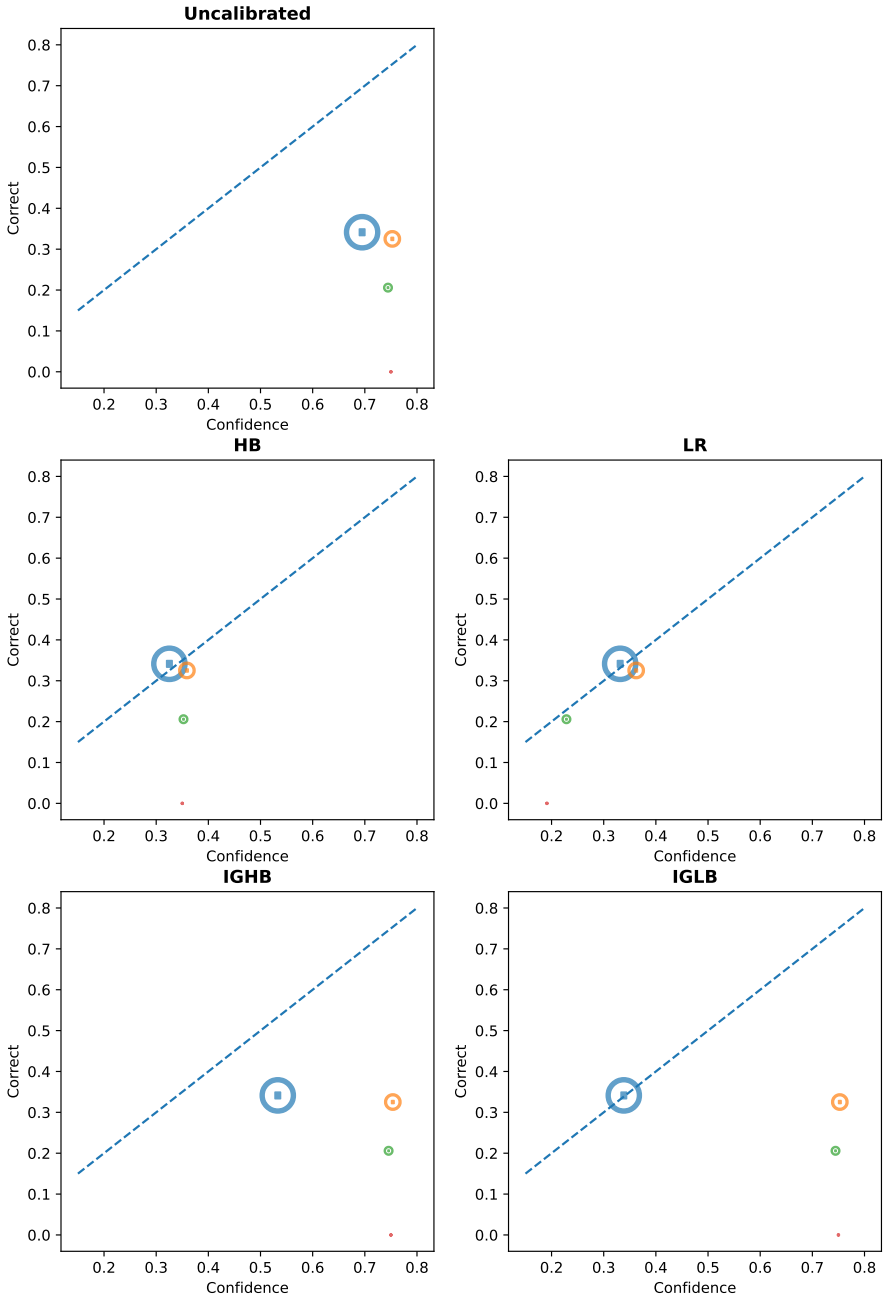


Figure 4.11: Diagram to compare the group calibration on the complexity grouping style for the baseline and each approach. Each circle represent a group. The diameter displays the number of samples in said groups. The groups are the follow: Cyclomatic complexity: 1-10 ●, Cyclomatic complexity: 1-20 ●, Cyclomatic complexity: 21-50 ●, Cyclomatic complexity: >50 ●

Categories

The group calibration results for the categories grouping style are displayed in Figure 4.12. We see that all groups are around the confidence of 70% in our baseline. And each group has roughly the same number of samples.

The colors of each group are assigned as follows:

- strings ●
- **not** strings ●
- mathematical ●
- **not** mathematical ●
- data objects ●
- **not** data objects ●

We can see in Figure 4.12 that the histogram binning only shifted the groups into a lower confidence. That made the calibration better because the groups are now closer to the perfect calibration line. But it did not manage to adjust the red and green group to perfect calibration.

The linear regression also shifted the confidence values to the lower values, but in contrast to the histogram binning all groups have a larger distance to the perfect calibration line. This means that the groups above the line are underestimated and the groups beneath the line are overestimated in their correctness.

Iterative group histogram binning did not manage to adjust any group. The problem can be that the max error is to high with the value of 0.05.

The iterative group linear binning on the other hand did some adjustments to the groups. But every group, except the green one, is far off from being well calibrated.

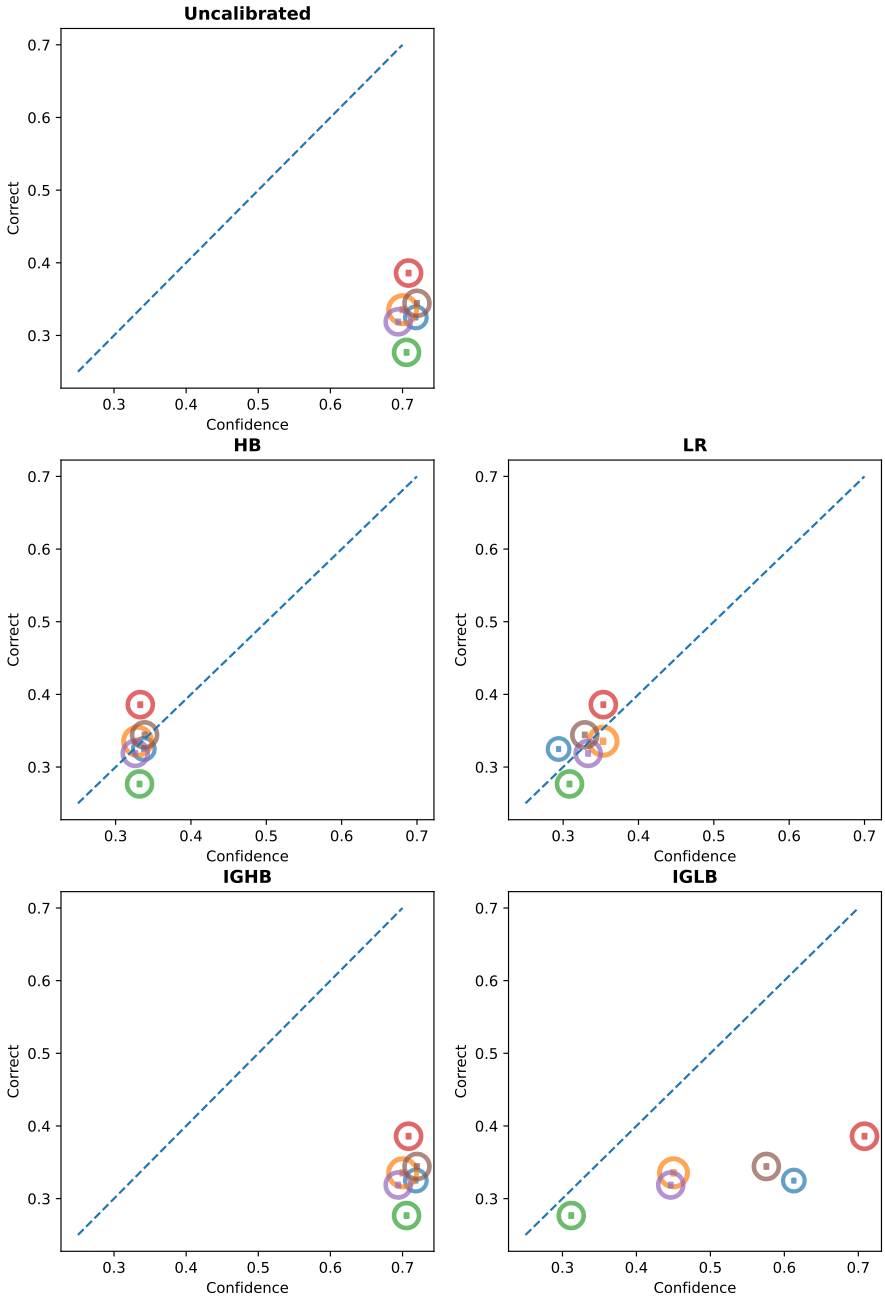


Figure 4.12: Diagram to compare the group calibration of the categories grouping style for the baseline and each approach. Each circle represent a group. The diameter displays the number of samples in said groups. The groups are the follow: strings ●, **not** strings ●, mathematical ●, **not** mathematical ●, data objects ●, **not** data objects ●

Metrics

To go into more detail we now look into the metrics for each grouping styles. Table 4.14 shows the metrics. The table is ordered by the calibration approaches. We highlighted the metrics bold for the grouping style that achieved best results for the calibration approach.

For the histogram binning we can see that the grouping style does not have any effect. This is not a surprise because groups were not used in this calibration approach.

The next calibration approach is the linear regression. We can see that the best ECE value was achieved with the complexity groups. But not far off is the simple grouping style with only a 0.04 higher ECE score. On the other scores the simple grouping style achieved the best or same results as the complexity groups.

Also the iterative group histogram binning approach achieved best value on the simple grouping style. Nevertheless the ECE values are the same for grouping style simple and complexity.

Our last approach the iterative group linear binning achieved the best score on all metrics with the simple grouping style.

This results indicate that the simple grouping style worked the best for all calibration approaches.

Grouping style	Method	ECE ↓	ASCE ↓	MSE ↓	$\mathbf{MSE_{ref}}$	Skill Score ↑
	Baseline	0.376	0.151	0.355	0.222	-0.599
-	HB	**0.036**	**0.003**	0.209	0.222	**0.059**
simple	LR	0.057	**0.004**	0.207	0.222	0.068
complexity	LR	**0.053**	**0.004**	0.208	0.222	0.063
categories	LR	0.061	0.005	0.209	0.222	0.059
combined	LR	**0.053**	0.005	**0.205**	0.222	**0.077**
simple	IGHB	**0.205**	**0.068**	**0.275**	0.222	**-0.239**
complexity	IGHB	**0.205**	0.096	0.302	0.222	-0.360
categories	IGHB	0.376	0.151	0.355	0.222	-0.599
combined	IGHB	0.217	0.077	0.285	0.222	-0.284
simple	IGLB	**0.101**	**0.033**	**0.243**	0.222	**-0.095**
complexity	IGLB	0.123	0.044	0.251	0.222	-0.131
categories	IGLB	0.186	0.061	0.260	0.222	-0.171
combined	IGLB	**0.101**	**0.033**	**0.243**	0.222	**-0.095**

Table 4.14: Shows the metrics for each approach we used with the different grouping styles. The bold scores indicates which grouping style provided the best results. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

The last metric we want to take a look at is the GASCE. This values is displayed for each group in the Table 4.15. In the Table 4.15 we see all groups and the association to their grouping style. The GASCE uncalibrated columns displays the GASCE values for the average token probabilities baseline. The bold score in each row indicates that this method achieved the best GASCE for the given group.

We can see that the histogram binning approach achieved the best GASCE the most of the time. The approach is followed by the linear regression approach which achieved same results as the histogram binning or sometime better values. Iterative group histogram binning on the other hand performed

worst of all four calibration approaches. The improved iterative group linear binning approach at least managed to achieve the same results as the histogram binning on three different groups.

Grouping style	Group	GASCE ↓				
		Uncalibrated	HB	LR	IGHB	IGLB
simple	> Median Prompt Len	0.09	**0.001**	**0.001**	0.042	0.022
simple	**not** > Median Prompt Len	0.065	0.005	**0.004**	0.030	0.017
simple	examples	0.038	0.003	**0.001**	0.033	0.038
simple	**not** examples	0.117	**0.003**	0.004	0.039	**0.003**
simple	> median code	0.089	**0.002**	**0.002**	0.037	0.023
simple	**not** > median code	0.065	**0.003**	0.004	0.034	0.016
complexity	1-10	0.106	**0.003**	0.005	0.052	**0.003**
complexity	11-20	0.034	**0.003**	**0.003**	0.034	0.034
complexity	21-50	0.014	**0.001**	0.002	0.014	0.014
complexity	>50	0.002	**0.**	**0.**	0.002	0.002
categories	strings	0.061	**0.002**	0.003	0.061	0.041
categories	**not** strings	0.093	**0.004**	0.006	0.093	0.027
categories	mathematical	0.098	**0.003**	0.004	0.098	**0.003**
categories	**not** mathematical	0.060	0.006	**0.005**	0.060	0.060
categories	data objects	0.078	**0.003**	0.006	0.078	0.023
categories	**not** data objects	0.077	**0.002**	0.003	0.077	0.043

Table 4.15: The table shows the GASCE for each group we used. The group column gives a short description of what the group describes. The lower the GASCE the better is the calibration in said group. The GASCE column display the GASCE values for every calibration approach. Bold values achieved best results on the corresponding group. The first column indicates the grouping style.

With our findings from the group calibration visualization and the different evaluated metrics, we can say that the simple grouping style provided the best results with our calibration approaches. Furthermore histogram binning and linear regression provided the best results for the group calibration.

4.6.1 No counter groups

The next experiment deals with the usage of counter groups. Counter groups are the exact opposite of a defined group. If a sample is not in the group „examples“it is assigned to the counter group „**not** examples“. This enables us to also target the samples which are not represented in a group and adjust their probabilities. For this experiment the **simple** grouping style is used.

The question arises:

RQ 4: Does the usage of counter groups improve the calibration results?

Table 4.16 shows the metrics with and without the usage of counter groups. The „yes “indicates that the method in the row used the counter groups. Analog the „no “means no counter groups were used. We can see that the usage of counter groups has no effect on the histogram binning and linear regression calibration approach. The scores stay the same for both variants.

The iterative group histogram binning and the iterative group linear binning on the other hand show an improvement on all scores when the counter groups are used. The ASCE has halved on both approaches.

Counter groups	Method	ECE ↓	ASCE ↓	MSE ↓	$\mathbf{MSE_{ref}}$	Skill Score ↑
	Baseline	0.376	0.151	0.355	0.222	-0.599
yes/no	HB	0.036	0.003	0.209	0.222	0.059
yes	LR	0.057	0.004	0.207	0.222	0.068
no	LR	0.057	0.004	0.207	0.222	0.068
yes	IGHB	**0.205**	**0.068**	**0.275**	0.222	**-0.239**
no	IGHB	0.376	0.151	0.355	0.222	-0.599
yes	IGLB	**0.101**	**0.033**	**0.243**	0.222	**-0.095**
no	IGLB	0.184	0.064	0.267	0.222	-0.203

Table 4.16: Shows the metrics for each approach with and without the counter groups. The bold scores indicate the best value for said metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

To conclude we can say that the counter groups had no impact on the calibration approaches histogram binning and linear regression. But the iterative group histogram binning and the iterative group linear binning achieved better results. This means that the counter groups improve the calibration results with certain calibration approaches.

4.7 Regressor

For our last experiment we tried to tweak our input features and experimented with different regressors. To recapitulate the linear regression approach 3.4.2 used the defined groups as input features and a bias b. The following used regressors will get the defined groups, the raw average token probability and a histogram, which represent the token probability distribution, as input features. The SVR was used with default settings from scikit learn and an RBF kernel. XGBoost was also used with default settings.

Table 4.17 shows the metrics that each regressor achieved. We see that the SVR performed worst on the ECE and ASCE metrics, but is better on the MSE than the XGBoost regressor. Both SVR and XGBoost have a negative skill score and therefore did not achieved better than the naive estimator MSE_{ref}.

The linear regression did perform better than the SVR and XGBoost on every metric. It even slightly outperformed the results from the histogram binning approach 4.10. Furthermore the skill score is almost 0.1 which means the regressor achieved better calibration results than the naive estimator MSE_{ref}.

Method	ECE ↓	ASCE ↓	MSE ↓	$\mathbf{MSE_{ref}}$	Skill Score ↑
Uncalibrated	0.376	0.151	0.355	0.222	-0.599
LR	**0.035**	**0.002**	**0.201**	0.222	**0.095**
SVR	0.196	0.044	0.245	0.222	-0.104
XGBoost	0.180	0.043	0.251	0.222	-0.131

Table 4.17: Shows the metrics for each regressor we used with the extended input features. The bold scores indicate the best value for said metrics. The arrows indicate if the score is better if it is closer to zero (↓) or better if it is higher (↑)

To conclude we can say that other regressor besides the linear regression did not perform better with regard to calibration. Furthermore the extension of the input features helped the linear regression to slightly outperform the histogram binning approach.

Chapter 5

Discussion

Sources

Literaturverzeichnis

[1] GLENN W. BRIER. Verification of forecasts expressed in terms of probability. Monthly Weather Review, 78(1):1 – 3, 1950.

[2] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. IEEE Transactions on Software Engineering, 49(7):3675–3691, 2023.

[3] Chih-Chung Chang and Chih-Jen Lin. Libsvm: A library for support vector machines. ACM Trans. Intell. Syst. Technol., 2(3), May 2011.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

[5] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16, page 785–794. ACM, August 2016.

[6] Gianluca Detommaso, Martin Bertran, Riccardo Fogliato, and Aaron Roth. Multicalibration for confidence scoring in llms, 2024.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the 29th Symposium on Operating Systems Principles, pages 611–626, 2023.

[8] Richard Oliver Lane. A comprehensive review of classifier probability calibration metrics, 2025.

[9] Fabrizio Le. How chatgpt is transforming the postdoc experience. Nature, 622:655, 2023.

[10] T.J. McCabe. A complexity measure. IEEE Transactions on Software Engineering, SE-2(4):308–320, 1976.

[11] Ansong Ni, Pengcheng Yin, Yilun Zhao, Martin Riddell, Troy Feng, Rui Shen, Stephen Yin, Ye Liu, Semih Yavuz, Caiming Xiong, Shafiq Joty, Yingbo Zhou, Dragomir Radev, Arman Cohan, and Arman Cohan. L2ceval: Evaluating language-to-code generation capabilities of large language models. Transactions of the Association for Computational Linguistics, 12:1311–1329, 10 2024.

[12] Mahdi Pakdaman Naeini, Gregory Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using bayesian binning. Proceedings of the AAAI Conference on Artificial Intelligence, 29(1), Feb. 2015.

[13] John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Adv. Large Margin Classif., 10, 06 2000.

[14] Matthew Renze and Erhan Guven. The effect of sampling temperature on problem solving in large language models, 2024.

[15] Paul J. Roebber. The regime dependence of degree day forecast technique, skill, and value. Weather and Forecasting, 13(3):783 – 794, 1998.

[16] Claudio Spiess, David Gros, Kunal Suresh Pai, Michael Pradel, Md Rafiqul Islam Rabin, Amin Alipour, Susmit Jha, Prem Devanbu, and Toufique Ahmed. Calibration and correctness of language models for code. arXiv preprint arXiv:2402.02047, 2024.

[17] Dennis Ulmer, Martin Gubri, Hwaran Lee, Sangdoo Yun, and Seong Oh. Calibrating large language models using their generations only. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 15440–15459, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[18] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. Expectation vs.nbsp;experience: Evaluating the usability of code generation tools powered by large language models. In Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems, CHI EA '22, New York, NY, USA, 2022. Association for Computing Machinery.

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[20] Yuvraj Virk, Premkumar Devanbu, and Toufique Ahmed. Enhancing trust in llm-generated code summaries with calibrated confidence scores. arXiv preprint arXiv:2404.19318, 2024.

[21] Stephen J Wright. Numerical optimization, 2006.

[22] Tong Xiao and Jingbo Zhu. Foundations of large language models, 2025.

[23] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. High-Confidence Computing, 4(2):100211, 2024.

[24] Li Zhong and Zilong Wang. Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 38, pages 21841–21849, 2024.

Online-Quellen

[25] BigCodeBench. Bigcodebench. [letzter Zugriff: 13. Jun. 2025].

[26] Ben Boyter. Sloc cloc and code (scc). [letzter Zugriff: 6. Jun. 2025].

[27] Thomas McCabe Jr. Software quality metrics to identify risk. [letzter Zugriff: 6. Jun. 2025].

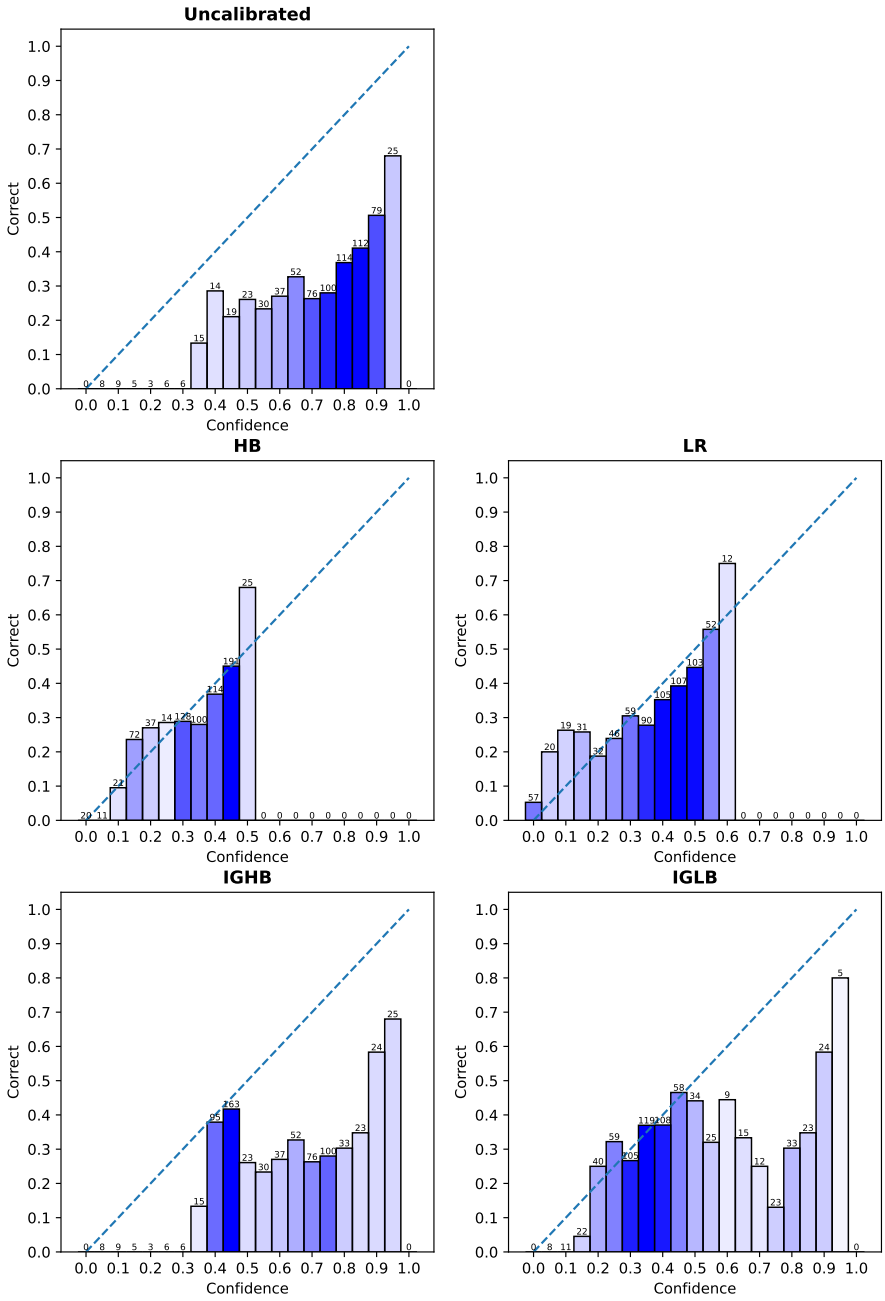
[28] Government of Canada. Probability calibration. [letzter Zugriff: 6. Jul. 2025].

[29] Amol Wagh. What's generative ai? explore underlying layers of machine learning and deep learning. [letzter Zugriff: 5. Jul. 2025].

[30] Deutscher Wetterdienst. Skill measure: Brier skill score. [letzter Zugriff: 6. Jul. 2025].

Appendix A

Charts



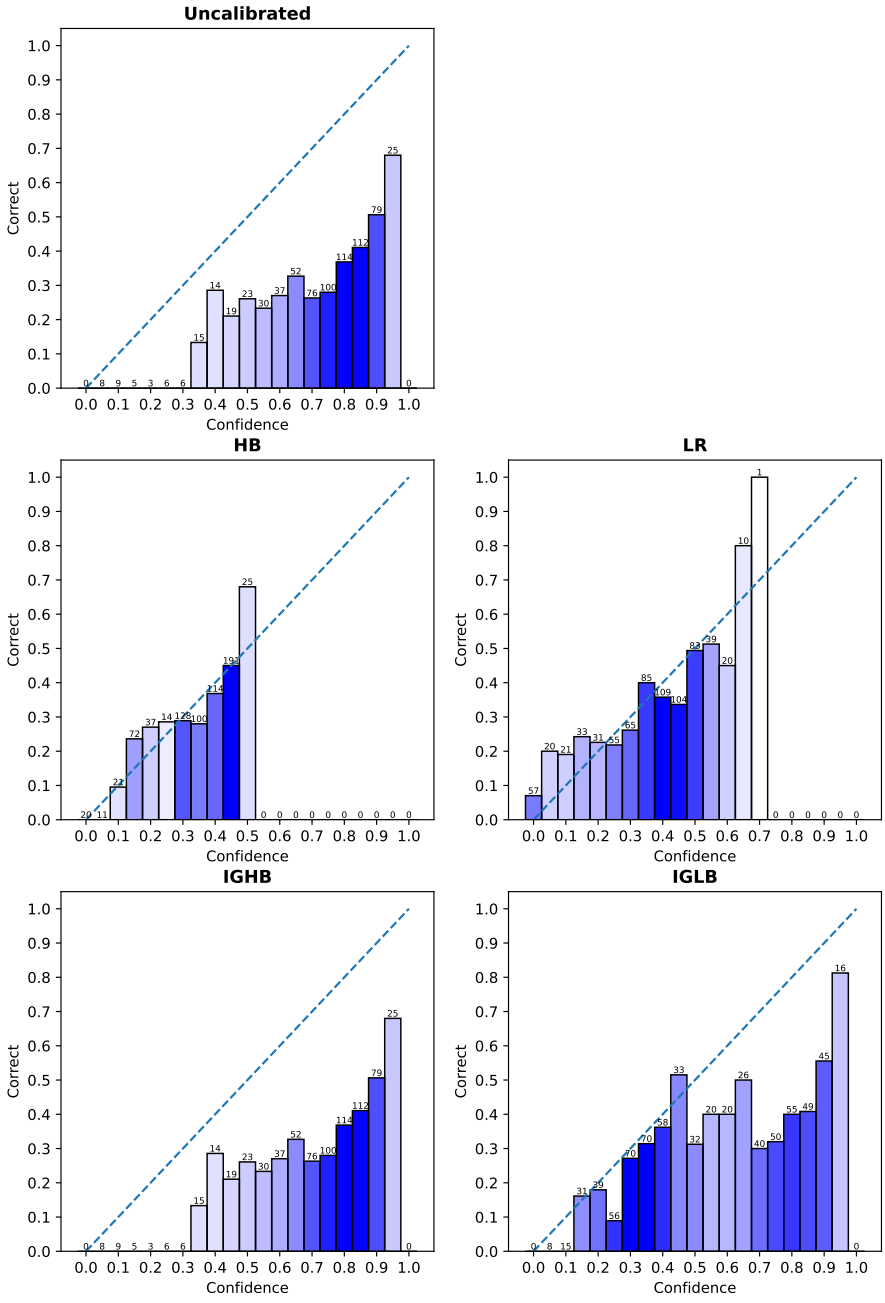

Uncalibrated
HB
LR
IGHB
IGLB
Correct
Confidence
0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0

Figure A.2

Appendix B

Prompts

B.1 Evaluator LLM

"Answer as short as you can. Here is a snippet of generated code consisting of the prompt, generated code and test cases: {code}"

Verbalized quantitative prompt: "Please provide your confidence in the correctness of the provided code only in percent (0-100 %): "

Verbalized qualitative prompt: "Please provide your confidence in the correctness of the provided code only as one of {Qualitative scale:}: "

Qualitative scale: "
{
"Very low": 0,
"Low": 0.15,
"Somewhat low": 0.3,
"Medium": 0.45,
"Somewhat high": 0.60,
"High": 0.75,
"Very high": 0.9,
} "